GUIA DO ALUNO

Monitor de água: pH, vazão e nível com Arduino e ESP32

pH-4502C · YF-S201 (efeito Hall) · HX710B (pressão) · ADS1115 (conversor A/D de 16 bits) — a química e a física de cada sensor, montagem no Uno, migração para o ESP32 e o caminho até a Rede de Telemetria do Espaço CRIA

Placas: Arduino Uno → ESP32 • **3 sensores + 1 conversor A/D + 1 projeto** • cada sensor: teste no Uno, depois no ESP32 com o celular

Projeto final: monitor de água com LED RGB e buzzer (alerta de pH, vazamento, consumo e transbordo), no Uno e no ESP32

Pré-requisitos: LEDs, LED RGB, buzzer, `millis()`, Monitor Serial e divisor de tensão (aulas anteriores)

Espaço Cria — 2026

Sumário

Como este guia funciona

Segurança: água, vidro e 3,3 V

Preparando o computador

Parte 1 — pH-4502C: a acidez da água

Parte 2 — YF-S201: vazão por efeito Hall

Parte 3 — HX710B: pressão e nível de água

Parte 4 — ADS1115: o conversor analógico-digital de 16 bits

Parte 5 — Projeto: monitor de água

Problemas comuns

Desafios

Referências

Como este guia funciona

Vamos construir um **monitor de água**: um aparelho que mede a **acidez (pH)**, a **vazão** (quantos litros por minuto passam por um cano) e o **nível** de um reservatório, e que avisa com LED e buzzer quando algo sai do normal — pH fora da faixa, torneira esquecida aberta (vazamento), consumo acima da meta, caixa transbordando. Cada sensor é testado **sozinho**, primeiro no **Arduino Uno** (5 V, simples) e depois no **ESP32** (3,3 V, Wi-Fi e página no celular). No meio do caminho estudamos um componente que não é sensor, mas que decide a qualidade de tudo: o **conversor analógico-digital ADS1115**, o mesmo usado nos nós da **Rede de Telemetria do Espaço CRIA**.

Parte	Peça	Mede	Princípio	Interface
1	pH-4502C + sonda BNC	Acidez (pH 0–14)	Eletrodo de vidro — equação de Nernst	Analógica 0–5 V (Po)
2	YF-S201 / ZJ-S201	Vazão (L/min) e volume (L)	Turbina + ímã + sensor de efeito Hall	Pulsos digitais (frequência)
3	HX710B + MPS20N0040D	Pressão (0–40 kPa) → nível de água (cm)	Membrana piezorresistiva em ponte de Wheatstone + A/D de 24 bits	Serial de 2 fios (OUT/SCK)
4	ADS1115	Tensão, com 16 bits	Conversor A/D sigma-delta com amplificador programável	I2C
5	Projeto	Tudo junto + LED RGB + buzzer	—	Uno: USB · ESP32: Wi-Fi

Onde estão os arquivos

Códigos em `aula-agua/codigo/`: `NN_sensor_uno` (Arduino), `NNb_sensor_esp32_web` (ESP32 + celular) e `01c_ph4502c_esp32_ads1115_web` (pH pelo ADS1115). Projeto: `04_monitor_agua_uno` e `04b_monitor_agua_esp32_web`.

Esquemas do Fritzing em `aula-agua/esquemas/saida/` (`.fzz` para abrir no Fritzing, `.png` para imprimir).

Todos os sketches usam o **Monitor Serial a 115200 baud** e aceitam **comandos** digitados (calibração). As leituras saem no formato `nome:valor`, que o **Plotter Serial** desenha.

Segurança: água, vidro e 3,3 V

- **Água longe das placas.** Só a ponta da sonda de pH, o corpo do YF-S201 (que é vedado) e a mangueira do sensor de pressão tocam a água. Arduino, ESP32, protoboard e os módulos ficam **acima** do nível da água e longe dos copos. Seque as mãos antes de mexer nos fios.
- **A sonda de pH é de vidro.** O bulbo na ponta é fino: não bata, não encoste no fundo do copo, não deixe secar. Entre uma medida e outra, enxágue com água destilada e **guarde na solução de armazenamento (KCl 3 M)**. Não deixe em água destilada por horas.
- **Pressão.** O YF-S201 aguenta até 1,75 MPa (17 atm) e o HX710B mede até 40 kPa (4 m de água). Na aula usamos só a pressão da torneira ou de um balde elevado: nada de compressor ou bomba de pneu no sensor de pressão — ele estoura com poucos metros de coluna de água.

- **ESP32 é 3,3 V.** O Po da placa de pH e os pulsos do YF-S201 saem a **5 V**: no ESP32 (e nas entradas do ADS1115 alimentado em 3,3 V) passam pelo **divisor 1 kΩ / 2 kΩ**. O HX710B pode ser alimentado em 3,3 V e dispensa o divisor.
- **Desligue a USB antes de mexer nos fios.** Os módulos de pH e de pressão têm amplificadores sensíveis: VCC e GND invertidos os destroem.

Preparando o computador

Só a Parte 4 precisa de biblioteca: **Adafruit ADS1X15** (Ferramentas → Gerenciar Bibliotecas; ela pede para instalar junto a **Adafruit BusIO**, aceite). O pH, o YF-S201 e o HX710B **não usam biblioteca**: fazemos as contas e o protocolo no próprio código, para entender o que acontece. No terminal (Linux, laboratório):

```terminal
arduino-cli lib install "Adafruit ADS1X15" "Adafruit BusIO"
arduino-cli compile --fqbn arduino:avr:uno      aula-agua/codigo/01_ph4502c_uno
arduino-cli compile --fqbn esp32:esp32:esp32    aula-agua/codigo/01b_ph4502c_esp32_web
arduino-cli upload -p /dev/ttyUSB0 --fqbn esp32:esp32:esp32
aula-agua/codigo/01b_ph4502c_esp32_web
```

Parte 1 — pH-4502C: a acidez da água

O que é e para que serve

A **placa pH-4502C** com **sonda de vidro BNC** mede o **pH de 0 a 14** (±0,1 depois de calibrada) de água de torneira, de aquário, de piscina, de solução hidropônica, de sucos, vinagre, leite, sabão… É o instrumento de qualquer laboratório de química e também o sensor que o nó `horta/no1` da telemetria usa para cuidar da solução nutritiva da bancada hidropônica.

A química: o que é pH e como um vidro o mede

Na água, uma fração minúscula das moléculas se separa em H^+ e OH^-. **$pH = -\log_{10}[H^+]$**: em água pura $[H^+] = 10^{-7}$ mol/L → pH 7. Cada unidade de pH é **10 vezes** mais íons H^+: vinagre (pH 3) tem 10 000 vezes mais H^+ que água pura. Ácido < 7 < básico (alcalino). Plantas absorvem bem os nutrientes entre pH 5,5 e 6,5; peixes de aquário gostam de 6,5–7,5; a água tratada chega à torneira entre 6,0 e 9,5.

A **sonda** é uma **célula eletroquímica**: um bulbo de **vidro especial** (silicato com lítio) separa a solução interna (pH 7 fixo) da amostra. Os íons H^+ das duas faces "trocam de lugar" com íons do vidro e geram uma **diferença de potencial** proporcional à diferença de pH — a **equação de Nernst**: E = −59,16 mV × (pH − 7) a 25 °C. Ou seja: **0 mV em pH 7; +177 mV em pH 4; −177 mV em pH 10**. Um segundo eletrodo (de referência, Ag/AgCl, no mesmo corpo) fecha o circuito. Esse sinal é minúsculo e de altíssima impedância (~100 MΩ): a placa 4502C tem um amplificador que o multiplica por ~3 e o desloca para **0–5 V, com pH 7 ≈ 2,5 V** — e é isso que lemos.

> **Por que calibrar sempre**
>
> Cada sonda tem seu **offset** (pH 7 não dá exatamente 2,5 V) e sua **inclinação** (mV por pH cai com a idade). A calibração de **dois pontos** (tampão pH 7 e pH 4 ou 10) resolve ambos: é uma **reta** V = V7 + inclinação × (pH − 7). Repita a calibração toda semana de uso.
>
> **Temperatura:** a inclinação de Nernst cresce 0,34 %/°C (59,16 mV a 25 °C; 54,2 mV a 0 °C). Entre 20 e 30 °C o erro é menor que 0,05 pH perto de pH 7 — desprezível para nós.

Montagem

Placa pH-4502C	Arduino Uno	ESP32 (ADC interno)	ESP32 + ADS1115 (Parte 4)
V+	5V	**VIN** (5 V)	VIN (5 V)
G (os dois)	GND	GND	GND
Po (saída de pH)	A0	1 kΩ → **GPIO 34** → 2 kΩ → GND	1 kΩ → **A0 do ADS1115** → 2 kΩ → GND
To (temperatura)	—	—	—
Do (limiar)	—	—	—
BNC	sonda	sonda	sonda

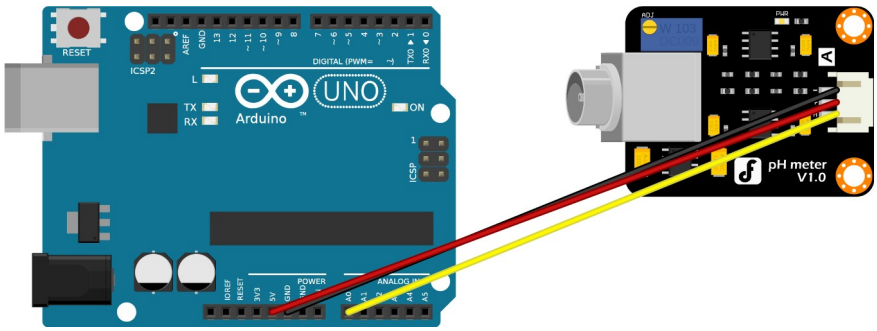


Figura 1 — Parte 1 — pH-4502C no Uno: V+ no 5V, G no GND, Po no A0. A sonda entra no BNC.

01_ph4502c_uno.ino

```
/*
  01 - pH-4502C + sonda BNC no Arduino Uno: medindo a acidez de liquidos
  Placa: Arduino Uno/Nano     Sem biblioteca: e um sinal analogico que NOS calibramos (EEPROM
guarda a calibracao)

  Placa pH-4502C:  V+ -> 5V    G -> GND (os dois G)    Po -> A0 (saida de pH)    To -> A1
(temperatura, opcional)
                   Do -> saida digital com limiar (pot pequeno; nao usamos)      A sonda entra no
conector BNC.
  Pot grande (perto do BNC) = OFFSET: ajusta a saida para 2,5 V com o BNC em curto (equivale a pH
7).
  Pot pequeno = limiar do Do (nao mexe no Po).

  Quimica: a sonda e uma celula eletroquimica. O bulbo de VIDRO especial troca ions H+ com o
liquido; a
  diferenca de concentracao de H+ entre a amostra e a solucao interna (pH 7) gera uma TENSAO
  (equacao de Nernst): -59,16 mV por unidade de pH a 25 C, 0 V em pH 7. pH 4 -> +177 mV; pH 10 ->
-177 mV.
  A placa amplifica (~x3) e desloca esse sinal para 0..5 V (pH 7 = 2,5 V). pH = -log10[H+].

  Calibracao de 2 pontos com solucoes tampao pH 7,00 e pH 4,00 (ou 10,00):
    1. Sonda em pH 7, espere estabilizar, digite  c7  no Monitor Serial -> grava V7
    2. Enxague com agua destilada, sonda em pH 4: digite  c4  -> grava a inclinacao (V/pH)
    r = volta ao padrao.  A inclinacao ideal e ~ -0,177 V/pH; abaixo de -0,15 (em modulo) a sonda
esta velha.
  Monitor Serial a 115200 baud. A linha "pH:7.02 V:2.503" tambem sai no Plotter Serial.
*/
#include <EEPROM.h>

const int PINO_PH = A0;
const int N_AMOSTRAS = 30;
const float VREF = 5.0;              // tensao real do 5V do Uno (meca com o multimetro e
corrija)
float V7 = 2.50;                     // tensao em pH 7 (offset)
float INCLINACAO = -0.177;           // V por unidade de pH (negativo: nesta placa, mais acido =
mais tensao)
const float PH_MIN_OK = 6.5, PH_MAX_OK = 8.5;   // faixa "agua boa" (aquario, piscina, torneira)

float lerTensao() {                  // media APARADA: 30 amostras, descarta as 5 maiores e as 5
menores
  int v[N_AMOSTRAS];
  for (int i = 0; i < N_AMOSTRAS; i++) { v[i] = analogRead(PINO_PH); delay(10); }
  for (int i = 0; i < N_AMOSTRAS - 1; i++)                      // ordena (bubble sort: 30
valores, tudo bem)
    for (int j = 0; j < N_AMOSTRAS - 1 - i; j++)
      if (v[j] > v[j + 1]) { int t = v[j]; v[j] = v[j + 1]; v[j + 1] = t; }
  long soma = 0;
  for (int i = 5; i < N_AMOSTRAS - 5; i++) soma += v[i];
  return (soma / float(N_AMOSTRAS - 10)) * VREF / 1023.0;
}

float tensaoParaPH(float v) { return 7.0 + (v - V7) / INCLINACAO; }   // a reta da calibracao

void salvar()   { EEPROM.put(0, V7); EEPROM.put(4, INCLINACAO); }
void carregar() {                    // so aceita valores plausiveis (EEPROM nova vem com lixo)
  float a, b; EEPROM.get(0, a); EEPROM.get(4, b);
  if (!isnan(a) && a > 1.0 && a < 4.0) V7 = a;
  if (!isnan(b) && b < -0.05 && b > -0.5) INCLINACAO = b;
```

```
}

void setup() {
  Serial.begin(115200);
  Serial.setTimeout(50);              // comandos chegam inteiros; sem isto readStringUntil
espera 1 s
  carregar();
  Serial.println(F("pH-4502C. Comandos: c7 (sonda em pH 7)  c4 (pH 4)  c10 (pH 10)  r (reset)"));
  Serial.print(F("V7 = ")); Serial.print(V7, 3); Serial.print(F(" V   inclinacao = "));
Serial.print(INCLINACAO, 4); Serial.println(F(" V/pH"));
}

void loop() {
  if (Serial.available()) {
    String cmd = Serial.readStringUntil('\n'); cmd.trim();
    float v = lerTensao();
    if (cmd == "c7") { V7 = v; salvar(); Serial.print(F("V7 gravado: ")); Serial.println(v, 3); }
    else if (cmd == "c4" || cmd == "c10") {
      float phRef = (cmd == "c4") ? 4.0 : 10.0;
      INCLINACAO = (v - V7) / (phRef - 7.0); salvar();
      Serial.print(F("inclinacao gravada: ")); Serial.print(INCLINACAO, 4); Serial.print(F(" V/pH
(saude da sonda: "));
      Serial.print(100.0 * INCLINACAO / -0.177, 0); Serial.println(F(" % da teorica)"));
    }
    else if (cmd == "r") { V7 = 2.5; INCLINACAO = -0.177; salvar(); Serial.println(F("calibracao
padrao restaurada")); }
  }
  float v = lerTensao(), ph = tensaoParaPH(v);
  const char* classe = ph < 6.5 ? "acido" : (ph > 7.5 ? "basico" : "neutro");
  Serial.print(F("pH:")); Serial.print(ph, 2); Serial.print(F(" V:")); Serial.print(v, 3);
  Serial.print(F("  ")); Serial.print(classe);
  if (ph < PH_MIN_OK || ph > PH_MAX_OK) Serial.print(F("  <<< fora da faixa 6,5-8,5"));
  Serial.println();
  delay(1000);
}
```

Entenda o código

- `lerTensao()` faz 30 leituras, **ordena, descarta as 5 maiores e as 5 menores** e tira a média do resto: é uma **média aparada**, robusta a ruído — o sinal de pH é muito sensível a interferência (a sonda é uma antena de 100 MΩ).
- `tensaoParaPH(v) = 7 + (v − V7) / INCLINACAO`: a reta da calibração. Inclinação padrão −0,177 V/pH (os 59 mV de Nernst × 3 do amplificador).
- Os comandos `c7`, `c4` e `c10` gravam os pontos na **EEPROM** (`EEPROM.put`): a calibração sobrevive ao desligar. `carregar()` só aceita valores plausíveis — uma EEPROM nova vem cheia de lixo.
- A "saúde da sonda" é a inclinação medida dividida pela teórica: abaixo de **85 %** a sonda está velha ou suja.

Calibração passo a passo

1. **Zerar o offset da placa (uma vez):** curto-circuite o BNC (um clipe entre o pino central e a carcaça) e gire o **potenciômetro grande** até `V:2.500`. Isso equivale a "pH 7 = 2,5 V".
2. Prepare as **soluções tampão** (sachês de pH 4,00 / 6,86 ou 7,00 / 9,18 ou 10,01 dissolvidos em 250 mL de água destilada). Duram ~1 mês fechadas.
3. Sonda em **pH 7**, espere o valor parar (1–2 min), digite `c7`.
4. Enxágue com água destilada, seque encostando (não esfregue), sonda em **pH 4**, espere, digite `c4`. A inclinação aparece: espere algo entre −0,15 e −0,19 V/pH.

5. Confira com o terceiro tampão (pH 10): erro ≤ 0,1. Se a inclinação estiver longe de −0,177, a sonda está velha ou o BNC está mal encaixado.
6. Meça amostras: vinagre (~2,5–3), suco de limão (~2), refrigerante (~2,5–3), água da torneira (~6,5–7,5), água com bicarbonato (~8,3), sabão diluído (~10).

Teste no ESP32 com o celular

O Po vai de 0 a 5 V; o ESP32 só aguenta 3,3 V. O **divisor 1 kΩ / 2 kΩ** entrega 2/3 da tensão (máximo 3,33 V). A calibração é feita **com o divisor montado** — ela absorve o fator 2/3 e as imperfeições do ADC do ESP32 (que são muitas, como você verá na Parte 4). Por isso os valores padrão no sketch são 1,667 V e −0,118 V/pH.

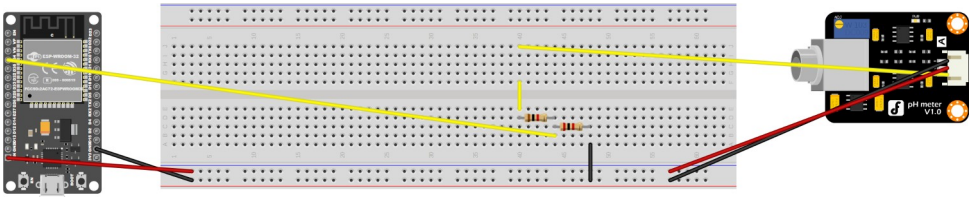


Figura 2 — Parte 1 — pH-4502C no ESP32: V+ no VIN (5 V), Po → 1 kΩ → GPIO 34 → 2 kΩ → GND (divisor na metade de baixo da protoboard).

O sketch do ESP32 cria a rede Wi-Fi **Agua-pH** (senha `12345678`). Conecte o celular a ela (desligue os dados móveis) e abra **http://192.168.4.1**: a página mostra o valor principal, os outros valores em cartões, um gráfico das últimas leituras e um aviso de **ALERTA**. Os arquivos `monitor_wifi.h` e `pagina_web.h` precisam estar **na mesma pasta** do sketch (a IDE os abre como abas).

01b_ph4502c_esp32_web.ino

```
/*
  01b - pH-4502C no ESP32 com monitor Wi-Fi no celular
  Placa: ESP32 Dev Module (DevKit V1, 30 pinos)    Sem biblioteca (Preferences guarda a
calibracao)
  Arquivos extras na mesma pasta: monitor_wifi.h, pagina_web.h

  Ligacoes: V+ -> VIN (5 V: a placa de pH precisa de 5 V)   G -> GND   Po -> 1 k -> GPIO 34 -> 2
k -> GND (DIVISOR!)
  O Po vai de 0 a 5 V; o divisor 1k/2k entrega 0 a 3,33 V ao ESP32 (maximo do pino: 3,6 V).
  Calibre COM o divisor montado: a calibracao absorve o fator 2/3 e a nao linearidade do ADC do
ESP32.
  Comandos pelo Monitor Serial (115200): c7, c4, c10, r - iguais ao sketch do Uno.
  Celular: rede "Agua-pH" (senha 12345678) -> http://192.168.4.1
*/
#include <Preferences.h>
#include "monitor_wifi.h"

const int PINO_PH = 34;                   // ADC1 (o ADC2 para de funcionar com o Wi-Fi ligado)
const int N_AMOSTRAS = 30;
const float V7_PADRAO = 1.667, INCL_PADRAO = -0.118;   // 2/3 de 2,5 V e de -0,177 V/pH (por
causa do divisor)
const float PH_MIN_OK = 6.5, PH_MAX_OK = 8.5;
float V7 = V7_PADRAO, INCLINACAO = INCL_PADRAO;
Preferences prefs;
unsigned long ultimaLeitura = 0;

float lerTensao() {                       // media aparada, em volts, ja com a calibracao de
fabrica do ADC
  int v[N_AMOSTRAS];
  for (int i = 0; i < N_AMOSTRAS; i++) { v[i] = analogReadMilliVolts(PINO_PH); delay(5); }
  for (int i = 0; i < N_AMOSTRAS - 1; i++)
    for (int j = 0; j < N_AMOSTRAS - 1 - i; j++)
      if (v[j] > v[j + 1]) { int t = v[j]; v[j] = v[j + 1]; v[j + 1] = t; }
  long soma = 0;
  for (int i = 5; i < N_AMOSTRAS - 5; i++) soma += v[i];
  return (soma / float(N_AMOSTRAS - 10)) / 1000.0;
}
```

```
float tensaoParaPH(float v) { return 7.0 + (v - V7) / INCLINACAO; }
void salvar()   { prefs.begin("ph", false); prefs.putFloat("v7", V7); prefs.putFloat("incl",
INCLINACAO); prefs.end(); }
void carregar() { prefs.begin("ph", true);  V7 = prefs.getFloat("v7", V7); INCLINACAO =
prefs.getFloat("incl", INCLINACAO); prefs.end(); }

void setup() {
  Serial.begin(115200);
  Serial.setTimeout(50);              // comandos chegam inteiros; sem isto readStringUntil
espera 1 s
  analogReadResolution(12);
  analogSetAttenuation(ADC_11db);         // ate ~3,1 V; confiavel entre 0,15 e 2,45 V
  carregar();
  Serial.printf("pH-4502C. V7 = %.3f V   inclinacao = %.4f V/pH   (c7, c4, c10, r)\n", V7,
INCLINACAO);
  monitorIniciar("Agua-pH");
}

void loop() {
  monitorAtender();                       // atende o celular sem parar
  if (Serial.available()) {
    String cmd = Serial.readStringUntil('\n'); cmd.trim();
    float v = lerTensao();
    if (cmd == "c7") { V7 = v; salvar(); Serial.printf("V7 gravado: %.3f V\n", v); }
    else if (cmd == "c4" || cmd == "c10") {
      INCLINACAO = (v - V7) / ((cmd == "c4" ? 4.0 : 10.0) - 7.0); salvar();
      Serial.printf("inclinacao: %.4f V/pH (saude da sonda: %.0f %% da teorica)\n", INCLINACAO,
100.0 * INCLINACAO / INCL_PADRAO);
    }
    else if (cmd == "r") { V7 = V7_PADRAO; INCLINACAO = INCL_PADRAO; salvar();
Serial.println("padrao restaurado"); }
  }
  if (millis() - ultimaLeitura < 1000) return;
  ultimaLeitura = millis();
  float v = lerTensao(), ph = tensaoParaPH(v);
  const char* classe = ph < 6.5 ? "acido" : (ph > 7.5 ? "basico" : "neutro");
  bool alerta = ph < PH_MIN_OK || ph > PH_MAX_OK;
  Serial.printf("pH:%.2f V:%.3f  %s%s\n", ph, v, classe, alerta ? "  <<< fora da faixa" : "");
  monitorPublicar(jsonMonitor("pH-4502C - acidez da agua",
    item("pH", ph, "") + "," + item("Tensao no pino", v, "V") + "," + item("V7 (calib.)", V7,
"V") + "," + item("Inclinacao", INCLINACAO * 1000, "mV/pH"),
    alerta, String("pH fora da faixa 6,5-8,5: ") + classe));
}
```

- `analogReadMilliVolts()` usa a calibração de fábrica do ADC do ESP32 e devolve milivolts: mais exato que `analogRead()`.
- A calibração vai para a **Preferences** (a “EEPROM” do ESP32, na flash).
- A página classifica ácido/neutro/básico e alerta fora de 6,5–8,5 (água de aquário).

Parte 2 — YF-S201: vazão por efeito Hall

O que é e para que serve

O **YF-S201** (também vendido como **ZJ-S201**) é um **hidrômetro** de plástico com rosca **G1/2"** (a de torneira de jardim) que mede **vazão de 1 a 30 L/min**. Com ele dá para saber quanto uma torneira gasta, detectar uma torneira esquecida aberta, dosar a água de uma horta, medir o consumo de um chuveiro. Na telemetria, ele mede a recirculação da bancada hidropônica (`horta/no1/vazao`).

A física: uma turbina, um ímã e o efeito Hall

Dentro do tubo há uma **turbina** (rotor) com um pequeno **ímã** numa das pás. A água empurra as pás e o rotor gira tanto mais rápido quanto maior a vazão. Do lado de fora, isolado da água, fica um **sensor de efeito Hall**: uma lâmina semicondutora percorrida por corrente. Quando o ímã passa, o campo magnético **B** empurra os elétrons para um lado da lâmina (**força de Lorentz**, F = q·v·B) e aparece uma **tensão de Hall** V_H = I·B / (n·q·t) entre as bordas. Um circuito dentro do sensor transforma essa tensão em **pulso digital**: o fio amarelo vai a 5 V e volta a 0 a cada volta do rotor.

Como cada volta corresponde a um certo volume de água que passou pelas pás, **o número de pulsos é proporcional ao volume** e **a frequência dos pulsos é proporcional à vazão**:

```
a conta do YF-S201
f (Hz) = K × Q (L/min)        K ≈ 7,5 para o YF-S201
pulsos por litro = K × 60 ≈ 450   →   cada pulso ≈ 2,2 mL
Q = 10 L/min → f = 75 Hz ;  Q = 1 L/min → f = 7,5 Hz
```

Característica	Valor	Observação
Faixa de vazão	1 a 30 L/min	abaixo de 1 L/min o rotor patina: pulsos irregulares
Pulsos por litro	~450 (K ≈ 7,5)	**±10 %** de peça para peça: por isso calibramos
Alimentação	5 a 18 V (mínimo 4,5 V), 15 mA	no ESP32: VIN (5 V), não o 3V3
Sinal	pulso de 0 ao VCC, ciclo de trabalho ~50 %	a 5 V precisa de divisor para o ESP32
Pressão máxima	1,75 MPa	a rede pública fica abaixo de 0,5 MPa
Fios	vermelho VCC · **amarelo sinal** · preto GND	há lotes com o amarelo no meio ou na ponta: siga as cores

Como contar pulsos: a interrupção

A 30 L/min chegam 225 pulsos por segundo, cada um com ~2 ms. Se o programa estivesse ocupado lendo o pH ou mandando texto pela serial, perderia pulsos — e perder pulsos é perder litros. A solução é a **interrupção externa**: o microcontrolador para o que estiver fazendo, executa uma função minúscula (`contarPulso()`, que só faz `pulsos++`) e volta ao ponto onde estava. No Uno só os pinos **2 e 3** têm essa capacidade; no ESP32, quase todos.

Três regras das interrupções

A variável compartilhada é **`volatile`**: avisa o compilador que ela muda "por fora" e que não pode ser guardada num registrador.

A função da interrupção é **curtíssima**: nada de `delay()`, `Serial.print()` ou contas de ponto flutuante dentro dela.

Para ler e zerar o contador, **desligue as interrupções por um instante** (`noInterrupts()` / `interrupts()` no Uno; `portENTER_CRITICAL` no ESP32). Senão um pulso pode chegar entre o "ler" e o "zerar" e sumir.

Montagem e teste no Arduino Uno

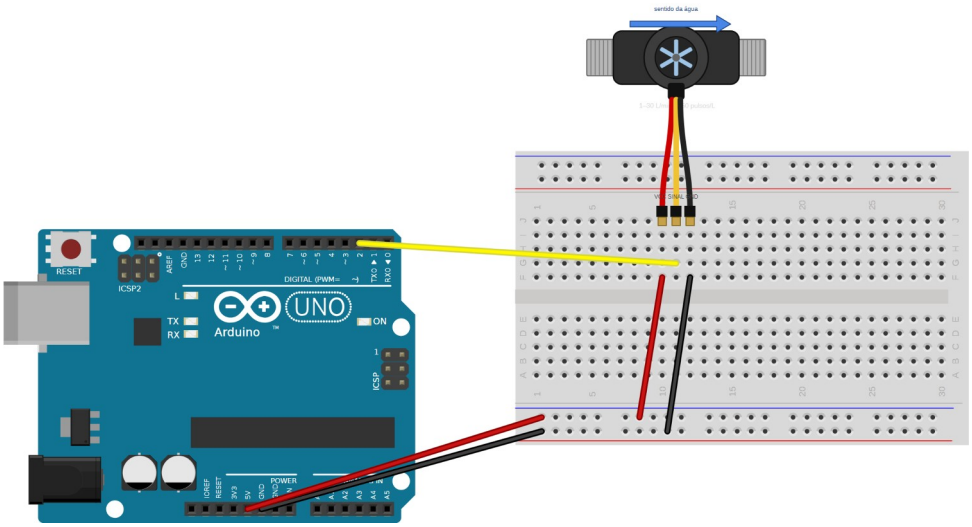


Figura 3 — Parte 2 — YF-S201 no Uno: vermelho no 5 V, amarelo no pino 2 (interrupção INT0), preto no GND. A seta do corpo indica o sentido da água.

02_yfs201_uno.ino

```
/*
  02 - Sensor de vazao YF-S201 / ZJ-S201 (efeito Hall) no Arduino Uno: litros por minuto e volume
total
  Placa: Arduino Uno/Nano     Sem biblioteca: contamos pulsos com uma INTERRUPCAO

  Ligacoes (cabo de 3 fios):  vermelho -> 5V    preto -> GND    amarelo (sinal) -> pino 2 (INT0)
  A agua deve entrar pelo lado da SETA marcada no corpo do sensor. Rosca G1/2" (a de torneira de
jardim).

  Fisica: dentro do tubo ha uma TURBINA com um ima. A agua gira a turbina; a cada volta o ima
passa
  por um SENSOR HALL, que gera um pulso. Mais vazao -> turbina mais rapida -> mais pulsos por
segundo:
     f (Hz) = K x Q (L/min)      K ~ 7,5 para o YF-S201  (450 pulsos por litro = 7,5 x 60)
  A frequencia e proporcional a vazao; o NUMERO de pulsos e proporcional ao VOLUME (cada pulso ~
2,2 mL).
  O fabricante diz +-10 %: por isso calibramos com um balde/garrafa de volume conhecido (comandos
b e f).

  Comandos no Monitor Serial (115200):
     z        zera o volume acumulado
     b        inicia a calibracao: encha um recipiente de volume conhecido...
     f 2.0    ...e termine informando o volume em litros (ex.: 2.0): o sketch calcula e grava o
novo K
     k        mostra o K atual        r  volta ao K padrao (7,5)
  Saida (tambem no Plotter Serial): Vazao_L_min:3.20 Freq_Hz:24.0 Volume_L:1.234
*/
#include <EEPROM.h>

const int PINO_SENSOR = 2;                  // INT0 (no Uno so os pinos 2 e 3 tem interrupcao
externa)
const float K_PADRAO = 7.5;                 // Hz por (L/min); pulsos por litro = K x 60
```

```
float K = K_PADRAO;

volatile unsigned long pulsos = 0;         // "volatile": e alterada DENTRO da interrupcao
unsigned long pulsosTotal = 0, pulsosCalibracao = 0;
float volumeL = 0;
bool calibrando = false;
unsigned long ultimoCalculo = 0;

void contarPulso() { pulsos++; }           // roda a cada borda de subida do sinal, mesmo no meio
do loop

void salvar()   { EEPROM.put(8, K); }                    // endereco 8: 0 e 4 sao do pH (sketch
01); o monitor 04 le daqui
void carregar() { float k; EEPROM.get(8, k); if (!isnan(k) && k > 3 && k < 15) K = k; }

void setup() {
  Serial.begin(115200);
  Serial.setTimeout(50);               // comandos chegam inteiros; sem isto readStringUntil
espera 1 s
  pinMode(PINO_SENSOR, INPUT_PULLUP);      // o sensor Hall ja puxa a linha; o pull-up evita
ruido com o cabo solto
  attachInterrupt(digitalPinToInterrupt(PINO_SENSOR), contarPulso, RISING);
  carregar();
  Serial.print(F("YF-S201: K = ")); Serial.print(K, 2); Serial.println(F(" Hz/(L/min). Comandos:
z, b, f <litros>, k, r"));
}

void loop() {
  if (Serial.available()) {
    String cmd = Serial.readStringUntil('\n'); cmd.trim();
    if (cmd == "z") { volumeL = 0; pulsosTotal = 0; Serial.println(F("volume zerado")); }
    else if (cmd == "b") { calibrando = true; pulsosCalibracao = 0; Serial.println(F("calibrando:
encha o recipiente e depois digite  f <litros>")); }
    else if (cmd.startsWith("f") && calibrando) {
      float litros = cmd.substring(1).toFloat();
      if (litros > 0 && pulsosCalibracao > 50) {
        K = (pulsosCalibracao / litros) / 60.0;       // pulsos por litro / 60 = Hz por (L/min)
        salvar(); calibrando = false;
        Serial.print(pulsosCalibracao); Serial.print(F(" pulsos em ")); Serial.print(litros, 3);
        Serial.print(F(" L  ->  K = ")); Serial.print(K, 3); Serial.println(F(" gravado na
EEPROM"));
      } else Serial.println(F("informe o volume (ex.: f 2.0) com pelo menos 50 pulsos
contados"));
    }
    else if (cmd == "k") { Serial.print(F("K = ")); Serial.print(K, 3); Serial.print(F("  ("));
Serial.print(K * 60, 0); Serial.println(F(" pulsos/L)")); }
    else if (cmd == "r") { K = K_PADRAO; salvar(); Serial.println(F("K padrao restaurado")); }
  }

  if (millis() - ultimoCalculo < 1000) return;      // uma conta por segundo
  unsigned long agora = millis();
  float dt = (agora - ultimoCalculo) / 1000.0;
  ultimoCalculo = agora;

  noInterrupts();                                    // copia e zera o contador sem a interrupcao
atrapalhar
  unsigned long n = pulsos; pulsos = 0;
  interrupts();

  float freq = n / dt;                               // Hz
  float vazao = freq / K;                            // L/min
  float litrosAgora = n / (K * 60.0);                // volume que passou neste intervalo
  volumeL += litrosAgora; pulsosTotal += n;
  if (calibrando) pulsosCalibracao += n;

  Serial.print(F("Vazao_L_min:")); Serial.print(vazao, 2);
  Serial.print(F(" Freq_Hz:")); Serial.print(freq, 1);
  Serial.print(F(" Volume_L:")); Serial.print(volumeL, 3);
  if (calibrando) { Serial.print(F("  [calibrando: ")); Serial.print(pulsosCalibracao);
Serial.print(F(" pulsos]")); }
  Serial.println();
}
```

Entenda o código

- `attachInterrupt(digitalPinToInterrupt(2), contarPulso, RISING)`: a cada **borda de subida** do pino 2, `contarPulso()` roda.
- Uma vez por segundo o `loop()` copia e zera `pulsos` com as interrupções desligadas, calcula `freq = n/dt`, `vazao = freq/K` e soma o volume `n/(K×60)`.
- `INPUT_PULLUP` mantém o pino em nível alto quando o cabo está solto, para o Uno não contar ruído como água.
- O **K** fica na EEPROM (comandos `b` e `f`): a calibração pelo balde, a seguir.

Roteiro de teste e calibração pelo balde

1. Sem água: `Vazao_L_min:0.00` e nenhum pulso. Sopre no tubo: o rotor gira e aparecem pulsos (sim, ele mede ar também — só que sem a calibração certa).
2. Ligue o sensor numa torneira com uma mangueira (ou despeje água de um balde elevado por uma mangueira). Abra devagar: a vazão sobe de 1 a 10 L/min. Veja o gráfico no Plotter Serial.
3. **Calibração:** digite `b`, encha uma garrafa PET de **2 L** (ou um balde graduado) passando pelo sensor, feche a torneira e digite `f 2.0`. O sketch calcula K = pulsos / litros / 60 e grava. Repita duas vezes: os valores devem bater em ±2 %.
4. Compare o seu K com os 7,5 do fabricante: a diferença é o erro que você teria sem calibrar.
5. Digite `z` e deixe uma torneira pingando por 1 minuto: o sensor detecta? (Não: abaixo de 1 L/min o rotor não gira. Limitação importante para um detector de vazamento.)

Teste no ESP32

O sensor Hall precisa de pelo menos 4,5 V, então é alimentado pelo **VIN**; o pulso sai a 5 V e passa pelo **divisor 1 kΩ / 2 kΩ** antes do **GPIO 27**. No ESP32 a função da interrupção leva `IRAM_ATTR` (fica na RAM, para rodar mesmo durante uma escrita na flash) e o contador é protegido por um **mutex de porta** (`portENTER_CRITICAL`). O nó `horta/no1` da telemetria usa o mesmo GPIO 27, mas com um **conversor de nível lógico** no lugar do divisor.

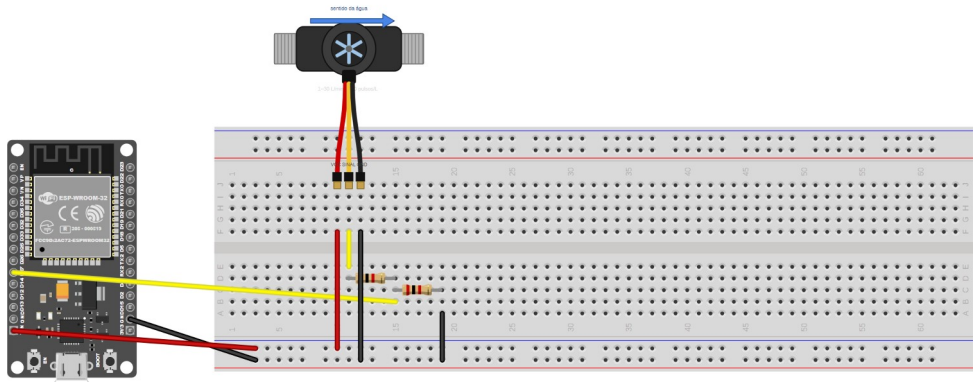


Figura 4 — Parte 2 — YF-S201 no ESP32: vermelho no VIN (5 V), amarelo → 1 kΩ → GPIO 27 → 2 kΩ → GND, preto no GND.

O sketch do ESP32 cria a rede Wi-Fi **Agua-Vazao** (senha `12345678`). Conecte o celular a ela (desligue os dados móveis) e abra **http://192.168.4.1**: a página mostra o valor principal, os outros valores em cartões, um gráfico das últimas leituras e um aviso de **ALERTA**. Os arquivos `monitor_wifi.h` e `pagina_web.h` precisam estar **na mesma pasta** do sketch (a IDE os abre como abas).

```
02b_yfs201_esp32_web.ino
```

```
/*
  02b - Sensor de vazao YF-S201 / ZJ-S201 no ESP32 com monitor Wi-Fi no celular
  Placa: ESP32 Dev Module     Sem biblioteca (Preferences guarda o K)     Arquivos extras:
monitor_wifi.h, pagina_web.h

  Ligacoes: vermelho -> VIN (5 V)    preto -> GND    amarelo -> 1 k -> GPIO 27 -> 2 k -> GND
(DIVISOR!)
  O sensor Hall e alimentado em 5 V e devolve pulsos de 5 V; o divisor 1k/2k os reduz a 3,3 V.
  (Alimentar o sensor em 3,3 V nao e garantido: a folha de dados pede no minimo 4,5 V.)
  Comandos pelo Serial (115200): z (zera volume), b / f <litros> (calibra o K), k, r.
  Celular: rede "Agua-Vazao" (senha 12345678) -> http://192.168.4.1
*/
#include <Preferences.h>
#include "monitor_wifi.h"

const int PINO_SENSOR = 27;
const float K_PADRAO = 7.5;                  // Hz por (L/min)
const float VAZAO_MAX_OK = 12.0;             // L/min: acima disso, "torneira escancarada"
(alerta)
float K = K_PADRAO;

volatile unsigned long pulsos = 0;
portMUX_TYPE mux = portMUX_INITIALIZER_UNLOCKED;   // "cadeado" para ler/zerar o contador com
seguranca
unsigned long pulsosCalibracao = 0, ultimoCalculo = 0;
float volumeL = 0, vazao = 0, freq = 0;
bool calibrando = false;
Preferences prefs;

void IRAM_ATTR contarPulso() { portENTER_CRITICAL_ISR(&mux); pulsos++;
portEXIT_CRITICAL_ISR(&mux); }

void salvar()   { prefs.begin("vazao", false); prefs.putFloat("k", K); prefs.end(); }
void carregar() { prefs.begin("vazao", true);  K = prefs.getFloat("k", K_PADRAO); prefs.end(); if
(K < 3 || K > 15) K = K_PADRAO; }

void setup() {
  Serial.begin(115200);
  Serial.setTimeout(50);               // comandos chegam inteiros; sem isto readStringUntil
espera 1 s
  pinMode(PINO_SENSOR, INPUT);               // o divisor 2 k ja segura o pino em 0 V quando nao
ha pulso
  attachInterrupt(digitalPinToInterrupt(PINO_SENSOR), contarPulso, RISING);
  carregar();
  Serial.printf("YF-S201: K = %.3f Hz/(L/min). Comandos: z, b, f <litros>, k, r\n", K);
  monitorIniciar("Agua-Vazao");
}

void loop() {
  monitorAtender();
  if (Serial.available()) {
    String cmd = Serial.readStringUntil('\n'); cmd.trim();
    if (cmd == "z") { volumeL = 0; Serial.println("volume zerado"); }
    else if (cmd == "b") { calibrando = true; pulsosCalibracao = 0; Serial.println("calibrando:
encha o recipiente e digite  f <litros>"); }
    else if (cmd.startsWith("f") && calibrando) {
      float litros = cmd.substring(1).toFloat();
      if (litros > 0 && pulsosCalibracao > 50) { K = (pulsosCalibracao / litros) / 60.0;
salvar(); calibrando = false; Serial.printf("%lu pulsos em %.3f L -> K = %.3f gravado\n",
pulsosCalibracao, litros, K); }
      else Serial.println("informe o volume (ex.: f 2.0) com pelo menos 50 pulsos");
    }
    else if (cmd == "k") Serial.printf("K = %.3f (%.0f pulsos/L)\n", K, K * 60);
    else if (cmd == "r") { K = K_PADRAO; salvar(); Serial.println("K padrao restaurado"); }
  }

  if (millis() - ultimoCalculo < 1000) return;
  unsigned long agora = millis();
  float dt = (agora - ultimoCalculo) / 1000.0;
  ultimoCalculo = agora;

  portENTER_CRITICAL(&mux);
  unsigned long n = pulsos; pulsos = 0;
  portEXIT_CRITICAL(&mux);
```

```
  freq = n / dt;
  vazao = freq / K;
  volumeL += n / (K * 60.0);
  if (calibrando) pulsosCalibracao += n;
  bool alerta = vazao > VAZAO_MAX_OK;
  Serial.printf("Vazao_L_min:%.2f Freq_Hz:%.1f Volume_L:%.3f%s\n", vazao, freq, volumeL,
calibrando ? "  [calibrando]" : "");
  monitorPublicar(jsonMonitor("YF-S201 - vazao de agua",
    item("Vazao", vazao, "L/min") + "," + item("Volume", volumeL, "L") + "," + item("Frequencia",
freq, "Hz") + "," + item("K", K, "Hz/(L/min)"),
    alerta, "vazao muito alta"));
}
```

Parte 3 — HX710B: pressão e nível de água

O que é e para que serve

O módulo **HX710B** junta duas peças: um **sensor de pressão MPS20N0040D** (0 a **40 kPa**, manométrica — isto é, acima da pressão atmosférica) e um **conversor analógico-digital de 24 bits HX710B**, primo do HX711 das balanças. Com uma mangueira, ele mede a pressão no fundo de um reservatório e, por uma conta simples, o **nível da água**: é o princípio dos sensores de nível de caixa-d'água e de máquinas de lavar (o "pressostato").

A física 1: pressão de uma coluna de água

Mergulhe um tubo com a ponta aberta até o fundo do reservatório, com a outra ponta ligada ao bico do sensor. O ar fica preso dentro do tubo e é comprimido pela água até ter **a mesma pressão que a água na ponta do tubo** — é o chamado **tubo de bolha**. E a pressão de uma coluna de líquido parado é a **lei de Stevin**:

```
nível a partir da pressão
P = ρ × g × h          ρ (água) = 1000 kg/m³    g = 9,81 m/s²
1 cm de água  =  1000 × 9,81 × 0,01  =  98,1 Pa
40 kPa  =  4,08 m de água     (o fundo de escala do sensor)
h (cm) = P (Pa) / 98,1
```

Repare que a pressão **não depende da largura** do reservatório nem do formato: só da altura da coluna. Um copo e uma caixa-d'água com 20 cm de água dão os mesmos 1962 Pa no fundo.

A física 2: piezorresistividade e a ponte de Wheatstone

O MPS20N0040D tem uma **membrana de silício** finíssima. Nela foram "impressos" quatro **resistores piezorresistivos**: quando o silício é esticado ou comprimido, o espaçamento entre seus átomos muda e a resistência também (é a **piezorresistividade**, muito mais forte no silício do que num fio metálico). Os quatro resistores formam uma **ponte de Wheatstone**: alimentada em 5 V, ela dá **0 V** quando os quatro são iguais (sem pressão) e uma tensão de **alguns milivolts** proporcional à pressão quando dois esticam e dois comprimem. A ponte é o truque que transforma uma variação de resistência minúscula numa tensão que dá para medir.

A física 3: por que 24 bits

Dezenas de milivolts para 40 kPa significa **~1 µV por pascal**, e 1 cm de água são 98 Pa, ou seja, uns 0,1 mV. O ADC do Uno (10 bits, 5 V) tem degraus de **4,9 mV** — não enxergaria nem 40 cm de água. O HX710B tem um **amplificador de ganho 128** e um conversor de **24 bits** (16,7 milhões de níveis): o degrau fica em dezenas de **nanovolts**. Na prática o ruído limita a resolução a uns 0,1 cm, o que já é excelente. O conversor conversa com o microcontrolador por um protocolo de 2 fios:

Sinal	O que acontece
OUT (DOUT) vai a 0	uma conversão ficou pronta (10 por segundo)
25 pulsos no SCK	a cada pulso sai 1 bit pelo OUT, do mais significativo ao menos (24 bits); o 25º pulso pede a próxima leitura da entrada diferencial a 10 Hz
26 ou 27 pulsos	26: a próxima leitura é a tensão DVDD−AVDD (diagnóstico); 27: entrada

Sinal	O que acontece
	diferencial a 40 Hz
SCK em 1 por mais de 60 µs	o chip dorme (power-down); por isso os pulsos são feitos com as interrupções desligadas
Número de 24 bits	complemento de 2: se o bit 23 for 1, o número é negativo (o sketch estende o sinal)

Montagem e calibração no Uno

Pino do módulo	Uno	ESP32
VCC	5V	**3V3** (o HX710B aceita 2,6–5,5 V; em 3,3 V o OUT já sai no nível do ESP32)
OUT (DOUT)	pino 3	GPIO 18
SCK	pino 4	GPIO 19
GND	GND	GND

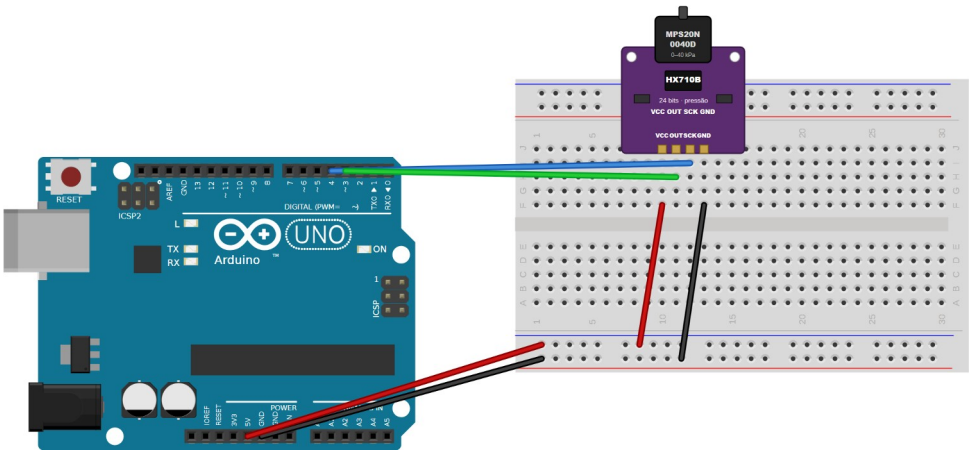


Figura 5 — Parte 3 — HX710B no Uno: VCC no 5 V, OUT no pino 3, SCK no pino 4. A mangueira vai do bico do sensor até o tubo mergulhado no reservatório.

03_hx710b_uno.ino

```
/*
  03 - Sensor de pressao HX710B + MPS20N0040D no Arduino Uno: pressao e NIVEL de agua
  Placa: Arduino Uno/Nano     Sem biblioteca: falamos o protocolo do HX710B com dois pinos


  Modulo (4 pinos):  VCC -> 5V    OUT -> pino 3    SCK -> pino 4    GND -> GND
  Mangueira: do bico do sensor ate um tubo com a ponta aberta mergulhado no reservatorio (tubo de
bolha).
  O ar preso no tubo fica na mesma pressao da agua na ponta: P = rho x g x h (rho = 1000 kg/m3, g
= 9,81 m/s2)
  -> cada 1 cm de coluna de agua = 98,1 Pa.  Faixa do sensor: 0-40 kPa = ate ~4 m de agua.


  Fisica: o MPS20N0040D tem uma MEMBRANA de silicio com 4 resistores piezorresistivos em PONTE DE
WHEATSTONE.
  A pressao deforma a membrana, a resistencia muda e aparece uma tensao de poucos milivolts (ate
~50 mV).
  O HX710B e um conversor A/D de 24 BITS com amplificador (ganho 128): le esses milivolts e
entrega um numero
  inteiro (complemento de 2, -8 388 608 a +8 388 607) por um protocolo serial simples:
    - OUT fica em 0 quando ha uma leitura pronta (10 por segundo)
    - a cada pulso em SCK sai 1 bit (24 bits, do mais significativo ao menos)
```

```
    - um 25o pulso diz "proxima leitura: entrada diferencial, 10 Hz" (26 = tensao DVDD-AVDD; 27 =
diferencial a 40 Hz)
    - SCK em 1 por mais de 60 us coloca o chip em modo de descanso (power-down)


  Calibracao de 2 pontos (gravada na EEPROM):
    c0        tubo FORA da agua (pressao atmosferica) -> grava o ZERO
    c 20      ponta do tubo a 20 cm abaixo da superficie (meca com a regua) -> grava a
sensibilidade (contagens/cm)
    r         volta ao padrao
  Saida (Plotter Serial): Contagem:123456 Pressao_Pa:1962 Nivel_cm:20.0
*/
#include <EEPROM.h>


const int PINO_OUT = 3, PINO_SCK = 4;
const float SENS_PADRAO = 50000.0;        // contagens por cm de agua (ordem de grandeza tipica;
CALIBRE)
long zero = 0;                            // contagem com o tubo fora da agua
float sens = SENS_PADRAO;
const int N_MEDIA = 8;


long lerHX710B() {                        // devolve a contagem de 24 bits ou 0x7FFFFFFF se o
modulo nao responder
  unsigned long t0 = millis();
  while (digitalRead(PINO_OUT) == HIGH) { if (millis() - t0 > 300) return 0x7FFFFFFF; }
  long v = 0;
  noInterrupts();                         // os pulsos nao podem ser interrompidos (SCK alto > 60
us = power-down)
  for (int i = 0; i < 24; i++) {
    digitalWrite(PINO_SCK, HIGH); delayMicroseconds(1);
    v = (v << 1) | digitalRead(PINO_OUT);
    digitalWrite(PINO_SCK, LOW);  delayMicroseconds(1);
  }
  digitalWrite(PINO_SCK, HIGH); delayMicroseconds(1); digitalWrite(PINO_SCK, LOW);   // 25o pulso
  interrupts();
  if (v & 0x800000L) v |= 0xFF000000L;    // estende o sinal: numero negativo em complemento de 2
  return v;
}


float lerMedia() {                        // media de N leituras (10 por segundo -> ~0,8 s)
  float soma = 0; int ok = 0;
  for (int i = 0; i < N_MEDIA; i++) { long v = lerHX710B(); if (v != 0x7FFFFFFF) { soma += v; ok+
+; } }
  return ok ? soma / ok : NAN;
}


void salvar()   { EEPROM.put(12, zero); EEPROM.put(16, sens); }
void carregar() { long z; float s; EEPROM.get(12, z); EEPROM.get(16, s); if (z != -1 && z >
-8000000L && z < 8000000L) zero = z; /* -1 = EEPROM virgem */ if (!isnan(s) && s > 100 && s <
1e7) sens = s; }


void setup() {
  Serial.begin(115200);
  Serial.setTimeout(50);              // comandos chegam inteiros; sem isto readStringUntil
espera 1 s
  pinMode(PINO_OUT, INPUT); pinMode(PINO_SCK, OUTPUT); digitalWrite(PINO_SCK, LOW);
  carregar();
  Serial.println(F("HX710B. Comandos: c0 (tubo fora da agua)   c <cm> (tubo a <cm> de
profundidade)   r"));
  Serial.print(F("zero = ")); Serial.print(zero); Serial.print(F("   sens = "));
Serial.print(sens, 0); Serial.println(F(" contagens/cm"));
}


void loop() {
  if (Serial.available()) {
    String cmd = Serial.readStringUntil('\n'); cmd.trim();
    if (cmd == "c0") { float m = lerMedia(); if (!isnan(m)) { zero = (long)m; salvar();
Serial.print(F("zero gravado: ")); Serial.println(zero); } }
    else if (cmd.startsWith("c ")) {
      float cm = cmd.substring(2).toFloat(); float m = lerMedia();
      if (cm > 0 && !isnan(m) && fabs(m - zero) > 100) { sens = (m - zero) / cm; salvar();
Serial.print(F("sensibilidade gravada: ")); Serial.print(sens, 1); Serial.println(F("
contagens/cm")); }
      else Serial.println(F("use c <cm> com o tubo mergulhado (e o zero ja gravado)"));
    }
    else if (cmd == "r") { zero = 0; sens = SENS_PADRAO; salvar(); Serial.println(F("padrao
restaurado: faca c0 e c <cm>")); }
```

```
  }
  float m = lerMedia();
  if (isnan(m)) { Serial.println(F("HX710B nao responde: confira OUT, SCK e a alimentacao"));
delay(500); return; }
  float nivelCm = (m - zero) / sens;
  float pressaoPa = nivelCm * 98.1;                  // rho g h com h em cm
  Serial.print(F("Contagem:")); Serial.print((long)m);
  Serial.print(F(" Pressao_Pa:")); Serial.print(pressaoPa, 0);
  Serial.print(F(" Nivel_cm:")); Serial.println(nivelCm, 1);
}
```

Entenda o código

- `lerHX710B()` espera o OUT ir a 0 (com **tempo limite** de 300 ms, para um fio solto não travar o programa), dá 24 pulsos lendo um bit em cada, dá o 25º pulso e estende o sinal.
- A conta `nivel = (contagem − zero) / sens`: `zero` é a contagem com o tubo fora da água (pressão atmosférica) e `sens` são as contagens por centímetro — os dois vêm da calibração e vão para a EEPROM.
- `pressao = nivel × 98,1` é só a lei de Stevin de volta.

Roteiro de teste e calibração

1. Monte o **tubo de bolha**: uma mangueira de aquário (4–6 mm) no bico do sensor; a outra ponta presa a uma régua, fixada numa garrafa PET cortada ou num balde transparente, com a ponta aberta para baixo.
2. Tubo **fora da água**: digite `c0`. A contagem deve ficar estável (variando poucas centenas).
3. Encha o recipiente até a ponta do tubo ficar **20,0 cm** abaixo da superfície (meça com a régua) e digite `c 20`. Pronto: o sketch gravou a sensibilidade.
4. Esvazie aos poucos e compare `Nivel_cm` com a régua em 15, 10 e 5 cm: erro menor que 0,5 cm?
5. Sopre de leve no tubo (não no sensor!): a pressão sobe para milhares de Pa. Aperte a mangueira: ela sobe e desce. Cuidado: pressão demais estoura a membrana.
6. Deixe o sensor por 10 minutos: a contagem “anda” sozinha? É a **deriva térmica** da ponte (a resistência do silício muda com a temperatura): por isso o zero deve ser refeito antes de cada medida importante.

Teste no ESP32

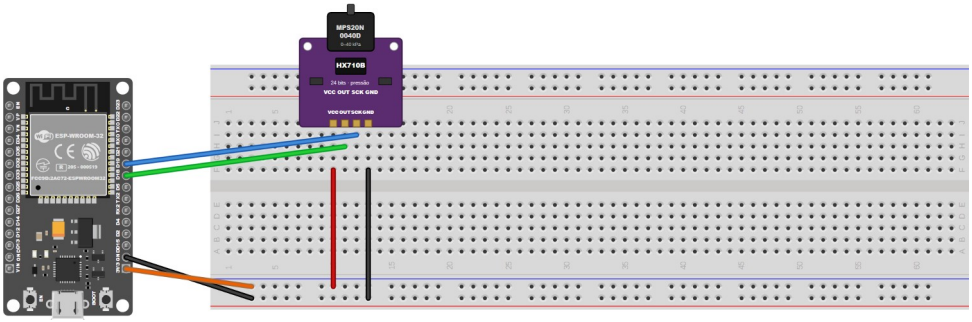


Figura 6 — Parte 3 — HX710B no ESP32: VCC no 3V3, OUT no GPIO 18, SCK no GPIO 19, sem divisor.

O sketch do ESP32 cria a rede Wi-Fi **Agua-Nivel** (senha `12345678`). Conecte o celular a ela (desligue os dados móveis) e abra **http://192.168.4.1**: a página mostra o valor principal, os outros valores em cartões,

um gráfico das últimas leituras e um aviso de **ALERTA**. Os arquivos `monitor_wifi.h` e `pagina_web.h` precisam estar **na mesma pasta** do sketch (a IDE os abre como abas).

03b_hx710b_esp32_web.ino

```
/*
  03b - Sensor de pressao HX710B + MPS20N0040D no ESP32 com monitor Wi-Fi: nivel do reservatorio
no celular
  Placa: ESP32 Dev Module     Sem biblioteca (Preferences)     Arquivos extras: monitor_wifi.h,
pagina_web.h

  Modulo: VCC -> 3V3 (o HX710B aceita 2,6-5,5 V; em 3,3 V o OUT ja sai no nivel do ESP32: SEM
divisor)
          OUT -> GPIO 18    SCK -> GPIO 19    GND -> GND
  Em 3,3 V a ponte do MPS20N0040D e excitada com 3,3 V (nao 5 V): a sensibilidade cai ~1/3 em
relacao ao Uno.
  Calibre (c0, c <cm>) NESTA montagem - zero e sens do sketch 03 (Uno, 5 V) nao valem aqui.
  Comandos (Serial 115200): c0 (zero com o tubo fora da agua), c <cm> (tubo a <cm> de
profundidade), r.
  Celular: rede "Agua-Nivel" (senha 12345678) -> http://192.168.4.1
*/
#include <Preferences.h>
#include "monitor_wifi.h"

const int PINO_OUT = 18, PINO_SCK = 19;
const float SENS_PADRAO = 50000.0;           // contagens/cm (ordem de grandeza; CALIBRE)
const float NIVEL_MIN_CM = 5.0, NIVEL_MAX_CM = 25.0;   // alertas: reservatorio vazio /
transbordando (ajuste ao seu recipiente)
long zero = 0; float sens = SENS_PADRAO;
Preferences prefs;
unsigned long ultimaLeitura = 0;

long lerHX710B() {
  unsigned long t0 = millis();
  while (digitalRead(PINO_OUT) == HIGH) { if (millis() - t0 > 300) return 0x7FFFFFFF; yield(); }
  long v = 0;
  portDISABLE_INTERRUPTS();                  // o Wi-Fi nao pode interromper os pulsos (SCK alto
> 60 us = power-down)
  for (int i = 0; i < 24; i++) {
    digitalWrite(PINO_SCK, HIGH); delayMicroseconds(1);
    v = (v << 1) | digitalRead(PINO_OUT);
    digitalWrite(PINO_SCK, LOW);  delayMicroseconds(1);
  }
  digitalWrite(PINO_SCK, HIGH); delayMicroseconds(1); digitalWrite(PINO_SCK, LOW);
  portENABLE_INTERRUPTS();
  if (v & 0x800000L) v |= 0xFF000000L;
  return v;
}
float lerMedia(int n = 4) { float soma = 0; int ok = 0; for (int i = 0; i < n; i++) { long v =
lerHX710B(); if (v != 0x7FFFFFFF) { soma += v; ok++; } } return ok ? soma / ok : NAN; }
void salvar()   { prefs.begin("nivel", false); prefs.putLong("zero", zero);
prefs.putFloat("sens", sens); prefs.end(); }
void carregar() { prefs.begin("nivel", true); zero = prefs.getLong("zero", 0); sens =
prefs.getFloat("sens", SENS_PADRAO); prefs.end(); if (sens < 100 || sens > 1e7) sens =
SENS_PADRAO; }

void setup() {
  Serial.begin(115200);
  Serial.setTimeout(50);               // comandos chegam inteiros; sem isto readStringUntil
espera 1 s
  pinMode(PINO_OUT, INPUT); pinMode(PINO_SCK, OUTPUT); digitalWrite(PINO_SCK, LOW);
  carregar();
  Serial.printf("HX710B. zero = %ld  sens = %.0f contagens/cm. Comandos: c0, c <cm>, r\n", zero,
sens);
  monitorIniciar("Agua-Nivel");
}

void loop() {
  monitorAtender();
  if (Serial.available()) {
    String cmd = Serial.readStringUntil('\n'); cmd.trim();
    if (cmd == "c0") { float m = lerMedia(8); if (!isnan(m)) { zero = (long)m; salvar();
Serial.printf("zero gravado: %ld\n", zero); } }
    else if (cmd.startsWith("c ")) {
      float cm = cmd.substring(2).toFloat(); float m = lerMedia(8);
      if (cm > 0 && !isnan(m) && fabs(m - zero) > 100) { sens = (m - zero) / cm; salvar();
```

```
Serial.printf("sensibilidade: %.1f contagens/cm\n", sens); }
      else Serial.println("use c <cm> com o tubo mergulhado (e o zero ja gravado)");
    }
    else if (cmd == "r") { zero = 0; sens = SENS_PADRAO; salvar(); Serial.println("padrao
restaurado"); }
  }
  if (millis() - ultimaLeitura < 1000) return;
  ultimaLeitura = millis();
  float m = lerMedia();
  if (isnan(m)) { Serial.println("HX710B nao responde"); monitorPublicar(jsonMonitor("HX710B",
item("Falha", 1, ""), true, "sensor nao responde")); return; }
  float nivel = (m - zero) / sens, pressao = nivel * 98.1;
  bool alerta = nivel < NIVEL_MIN_CM || nivel > NIVEL_MAX_CM;
  Serial.printf("Contagem:%ld Pressao_Pa:%.0f Nivel_cm:%.1f%s\n", (long)m, pressao, nivel, alerta
? "  <<< nivel fora da faixa" : "");
  monitorPublicar(jsonMonitor("HX710B - nivel do reservatorio",
    item("Nivel", nivel, "cm") + "," + item("Pressao", pressao, "Pa") + "," + item("Contagem", m,
"") + "," + item("Zero", zero, ""),
    alerta, nivel < NIVEL_MIN_CM ? "reservatorio quase vazio" : "transbordando!"));
}
```

- No ESP32 os 24 pulsos são feitos com `portDISABLE_INTERRUPTS()`: o Wi-Fi não pode interromper no meio e deixar o SCK em 1 por mais de 60 µs.
- **Calibre de novo no ESP32.** Em 3,3 V a ponte de Wheatstone do sensor é excitada com 3,3 V (no Uno eram 5 V): a tensão de saída, e portanto as contagens por centímetro, caem para ~2/3. O `zero` e a `sens` medidos no Uno **não valem** aqui — refaça `c0` e `c 20` nesta montagem (o mesmo vale para o projeto 5B).
- A página alerta **reservatório quase vazio** (< 5 cm) e **transbordando** (> 25 cm): ajuste `NIVEL_MIN_CM` e `NIVEL_MAX_CM` ao seu recipiente.

Parte 4 — ADS1115: o conversor analógico-digital de 16 bits

O que faz um conversor A/D

Um sensor analógico, como a placa de pH, entrega uma **tensão** que varia continuamente. O microcontrolador só entende **números**. O **conversor analógico-digital (ADC)** faz a ponte: a cada leitura ele compara a tensão com uma referência e devolve um inteiro. Dois números definem a qualidade dessa conversão: a **resolução** (quantos bits — quantos "degraus" há entre 0 e a referência) e a **exatidão** (se o degrau 2048 corresponde mesmo à metade da referência). O Uno tem um ADC de **10 bits** (1024 degraus de 4,9 mV, razoavelmente linear). O ESP32 tem um de **12 bits** (4096 degraus) que, infelizmente, é **não linear**, só é confiável entre ~0,15 e 2,45 V e sofre com o ruído do Wi-Fi — você viu isso na Parte 1, quando a calibração precisou absorver as imperfeições dele.

O ADS1115

O **ADS1115** (Texas Instruments) é um ADC **externo**, de **16 bits** (65 536 níveis, sendo 1 bit de sinal), **sigma-delta** (um tipo de conversor que troca velocidade por exatidão), com **4 entradas** (ou 2 pares diferenciais), um **amplificador de ganho programável (PGA)** e interface **I2C** — a mesma de displays e do relógio DS3231. Ele mede a tensão com exatidão e **manda o número pronto** pelo barramento: o ruído do fio não altera um número que viaja em bits. Custa pouco e vem num módulo com 10 pinos.

Ganho (PGA)	Fundo de escala	1 nível (LSB)	Quando usar
2/3	±6,144 V	0,1875 mV	sinal até 5 V (ADS1115 alimentado em 5 V) — nó `horta/no1`
1	±4,096 V	0,125 mV	sinal até 3,3 V — nosso sketch `01c`
2	±2,048 V	0,0625 mV	sinal pequeno
4 / 8 / 16	±1,024 / ±0,512 / ±0,256 V	31 / 15,6 / 7,8 µV	termopares, pontes… (o HX710B faz o mesmo, com ganho 128)

Dois limites que confundem todo mundo

O PGA **não aumenta a tensão que a entrada aguenta**: qualquer entrada deve ficar entre GND − 0,3 V e **VDD + 0,3 V**. Com o ADS1115 em 3,3 V, o Po da placa de pH (até 5 V) **precisa do divisor**, mesmo que o ganho 2/3 "chegue" a 6,144 V.

A resolução real é de **15 bits** para sinais positivos (o 16º bit é o sinal). Com ganho 1: 4,096 V / 32 768 = 0,125 mV.

	ADC do Uno	ADC do ESP32	ADS1115	HX710B
Bits	10	12	16	24
Níveis	1024	4096	65 536	16,7 milhões
Degrau (5 V / 3,3 V)	4,9 mV	0,8 mV	0,125 mV (ganho 1)	~nV (ganho 128)
Linearidade	boa	ruim (não linear, satura)	excelente	excelente
Entradas	6	muitas (ADC2 morre com Wi-Fi)	4 (ou 2 diferenciais)	1 diferencial

	ADC do Uno	ADC do ESP32	ADS1115	HX710B
Velocidade	~10 000/s	~100 000/s	8 a 860/s	10 ou 40/s
Interface	interna	interna	I2C	2 fios (própria)
Para que serve aqui	pH no Uno	pH e MQ no ESP32 (com calibração)	pH e condutividade na telemetria	pressão (ponte de mV)

Por que ele é importante para a Rede de Telemetria do Espaço CRIA

O nó **`horta/no1`** da telemetria mede a solução nutritiva da bancada hidropônica NFT e publica `ph`, `condutividade`, `temperatura`, `vazao`, `pressao`, `nivel` e `bomba`. As três grandezas **analógicas** (pH, condutividade e pressão) passam pelo **ADS1115**: o pH precisa de ±0,2 de exatidão para a planta absorver nutrientes, e a condutividade (quantidade de sais, em µS/cm) é uma leitura de milivolts que o ADC do ESP32 não entregaria com confiança — nem com o ruído do Wi-Fi ligado o tempo todo. Com o ADS1115 as calibrações ficam **estáveis e repetíveis**, e a ferramenta de calibração do notebook (`telemetria-cria/calibracao/`) grava as retas no nó.

No nó `horta/no1`	Como está ligado	Por quê
ADS1115	**VDD no 5 V**, ADDR no GND (0x48); SDA/SCL no lado de 5 V de um **conversor de nível lógico**; o lado de 3,3 V vai aos GPIO 21 (SDA) e 22 (SCL)	alimentado em 5 V ele lê o Po (até 5 V) e o sensor de pressão **direto**, sem divisor; mas o I2C fica em 5 V e o ESP32 não tolera — daí o conversor de nível
PH-4502C	Po → **A0** do ADS1115	ganho 2/3 (±6,144 V): 0,1875 mV por nível
Sensor TDS (condutividade)	sinal → **A1**; VCC no **GPIO 32**	o nó **desliga o TDS** fora da leitura: ele injeta corrente na água e desvia o pH
Sensor de pressão 0–1,2 MPa	sinal → **A2**	pressão da linha da bomba (kPa manométrica); erro ~12 kPa (1 % do fundo de escala)
YF-S201	amarelo → canal 3 do conversor de nível → **GPIO 27**	mesma função do nosso divisor, com borda mais limpa

Nesta aula fazemos a **versão simples**: ADS1115 alimentado em **3,3 V** (I2C já no nível do ESP32, sem conversor de nível) e o Po pelo **divisor**. O sketch desfaz o divisor (× 1,5) e mostra a tensão real do Po — pH 7 em ~2,5 V, igual ao Uno. Quando você montar o nó da horta, basta trocar a alimentação para 5 V, inserir o conversor de nível e usar o ganho 2/3.

ADS1115	ESP32	
VDD	3V3	2,0–5,5 V; consome 150 µA
GND	GND	
SCL	GPIO 22	I2C do projeto
SDA	GPIO 21	
ADDR	GND	endereço **0x48** (VDD = 0x49, SDA = 0x4A, SCL = 0x4B: até 4 módulos no

ADS1115	ESP32	
		mesmo barramento)
A0	divisor do Po	A1–A3 livres (condutividade, pressão analógica…)
ALRT	—	aviso “conversão pronta” ou comparador (não usamos)

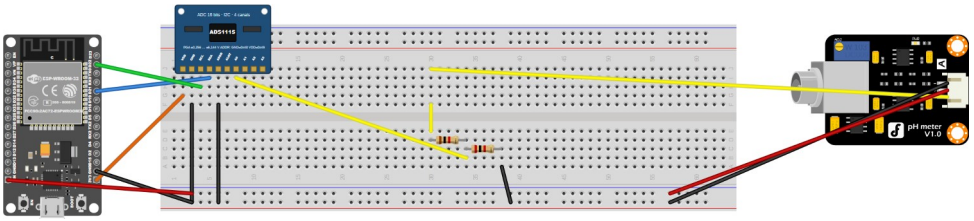


Figura 7 — Parte 4 — pH-4502C lido pelo ADS1115 no ESP32: ADS1115 em 3,3 V (VDD, GND, SCL→22, SDA→21, ADDR→GND), Po → divisor 1 kΩ/2 kΩ → A0.

O sketch do ESP32 cria a rede Wi-Fi **Agua-pH-ADS** (senha `12345678`). Conecte o celular a ela (desligue os dados móveis) e abra **http://192.168.4.1**: a página mostra o valor principal, os outros valores em cartões, um gráfico das últimas leituras e um aviso de **ALERTA**. Os arquivos `monitor_wifi.h` e `pagina_web.h` precisam estar **na mesma pasta** do sketch (a IDE os abre como abas).

01c_ph4502c_esp32_ads1115_web.ino

```
/*
  01c - pH-4502C no ESP32 lido pelo ADS1115 (conversor A/D de 16 bits, I2C) + monitor Wi-Fi
  Placa: ESP32 Dev Module     Biblioteca: "Adafruit ADS1X15" (instala junto a Adafruit BusIO)
  Arquivos extras: monitor_wifi.h, pagina_web.h

  Por que um A/D externo? O ADC interno do ESP32 tem 12 bits, e nao linear, so e confiavel entre
~0,15 e 2,45 V
  e sofre com o ruido do Wi-Fi. O ADS1115 tem 16 bits (65 536 niveis, 15 bits + sinal), e linear,
tem um
  amplificador de ganho programavel (PGA) e conversa por I2C: a leitura chega como NUMERO, nao
como tensao
  sujeita a ruido no fio. E o conversor que os nos da Rede de Telemetria do Espaco CRIA usam para
o pH e a
  condutividade (no horta/no1).

  Ligacoes (ADS1115 alimentado em 3,3 V, como o ESP32 -> I2C ja no nivel certo, sem conversor de
nivel):
    ADS1115   VDD -> 3V3   GND -> GND   SCL -> GPIO 22   SDA -> GPIO 21   ADDR -> GND (endereco
0x48)
    pH-4502C  V+ -> VIN (5 V)   G -> GND   Po -> 1 k -> A0 do ADS1115 -> 2 k -> GND  (divisor: as
entradas do
              ADS1115 nao podem passar de VDD + 0,3 V = 3,6 V; o Po vai ate 5 V)
  O sketch desfaz o divisor (x 1,5) e mostra a tensao REAL do Po: pH 7 = ~2,5 V, igual ao Uno.
  (No no horta/no1 da telemetria o ADS1115 e alimentado em 5 V, le o Po direto, e um conversor de
nivel
   logico protege o I2C do ESP32 - veja o guia.)
  Comandos (Serial 115200): c7, c4, c10, r.   Celular: rede "Agua-pH-ADS" -> http://192.168.4.1
*/
#include <Adafruit_ADS1X15.h>
#include <Preferences.h>
#include "monitor_wifi.h"

const uint8_t CANAL_PH = 0;
const float FATOR_DIVISOR = 1.5;          // (1 k + 2 k) / 2 k
const float V7_PADRAO = 2.50, INCL_PADRAO = -0.177;
const float PH_MIN_OK = 6.5, PH_MAX_OK = 8.5;
float V7 = V7_PADRAO, INCLINACAO = INCL_PADRAO;
Adafruit_ADS1115 ads;
bool adsPresente = false;
Preferences prefs;
unsigned long ultimaLeitura = 0;

float lerTensao() {                       // media de 8 conversoes (8 ms cada a 128 SPS); NAN se
```

```
o ADS1115 nao responder
  if (!adsPresente) return NAN;
  long soma = 0;
  for (int i = 0; i < 8; i++) {
    ads.startADCReading(MUX_BY_CHANNEL[CANAL_PH], false);
    unsigned long t0 = millis();
    while (!ads.conversionComplete()) { if (millis() - t0 > 25) return NAN; delay(1); }   // fio
solto nao trava o loop
    soma += ads.getLastConversionResults();
  }
  return ads.computeVolts((int16_t)(soma / 8)) * FATOR_DIVISOR;
}
float tensaoParaPH(float v) { return 7.0 + (v - V7) / INCLINACAO; }
void salvar()   { prefs.begin("phads", false); prefs.putFloat("v7", V7); prefs.putFloat("incl",
INCLINACAO); prefs.end(); }
void carregar() { prefs.begin("phads", true);  V7 = prefs.getFloat("v7", V7); INCLINACAO =
prefs.getFloat("incl", INCLINACAO); prefs.end(); }


void setup() {
  Serial.begin(115200);
  Serial.setTimeout(50);              // comandos chegam inteiros; sem isto readStringUntil
espera 1 s
  Wire.begin(21, 22);
  adsPresente = ads.begin(0x48, &Wire);
  if (adsPresente) {
    ads.setGain(GAIN_ONE);                // +-4,096 V de fundo de escala: 1 LSB = 0,125 mV (o
pino chega a 3,33 V)
    ads.setDataRate(RATE_ADS1115_128SPS);
    Serial.println("ADS1115 em 0x48: ok  (ganho 1: 0,125 mV por nivel)");
  } else Serial.println("ERRO: ADS1115 nao responde em 0x48 - confira SDA/SCL/VDD e o ADDR no
GND");
  carregar();
  Serial.printf("V7 = %.3f V   inclinacao = %.4f V/pH   (c7, c4, c10, r)\n", V7, INCLINACAO);
  monitorIniciar("Agua-pH-ADS");
}

void loop() {
  monitorAtender();
  if (Serial.available()) {
    String cmd = Serial.readStringUntil('\n'); cmd.trim();
    float v = lerTensao();
    if (isnan(v)) Serial.println("sem leitura do ADS1115");
    else if (cmd == "c7") { V7 = v; salvar(); Serial.printf("V7 gravado: %.3f V\n", v); }
    else if (cmd == "c4" || cmd == "c10") { INCLINACAO = (v - V7) / ((cmd == "c4" ? 4.0 : 10.0) -
7.0); salvar(); Serial.printf("inclinacao: %.4f V/pH (%.0f %% da teorica)\n", INCLINACAO, 100.0 *
INCLINACAO / INCL_PADRAO); }
    else if (cmd == "r") { V7 = V7_PADRAO; INCLINACAO = INCL_PADRAO; salvar();
Serial.println("padrao restaurado"); }
  }
  if (millis() - ultimaLeitura < 1000) return;
  ultimaLeitura = millis();
  float v = lerTensao();
  if (isnan(v)) { monitorPublicar(jsonMonitor("pH (ADS1115)", item("Falha", 1, ""), true,
"ADS1115 nao responde")); return; }
  float ph = tensaoParaPH(v);
  bool alerta = ph < PH_MIN_OK || ph > PH_MAX_OK;
  Serial.printf("pH:%.3f V_Po:%.4f V\n", ph, v);          // repare nas 4 casas: resolucao de
0,19 mV no Po
  monitorPublicar(jsonMonitor("pH-4502C pelo ADS1115",
    item("pH", ph, "") + "," + item("Tensao do Po", v, "V") + "," + item("V7 (calib.)", V7, "V")
+ "," + item("Inclinacao", INCLINACAO * 1000, "mV/pH"),
    alerta, "pH fora da faixa 6,5-8,5"));
}
```

Entenda o código

- `ads.begin(0x48, &Wire)` procura o chip no barramento; se não responder, o sketch avisa e segue (sem travar o Wi-Fi).
- `setGain(GAIN_ONE)`: ±4,096 V; `setDataRate(128 SPS)`: cada conversão leva ~8 ms. `startADCReading` + `conversionComplete` é a leitura **sem bloquear** usada no firmware `horta-no1`, com **prazo de 25 ms** — um fio solto no I2C não pode travar o nó.

- `computeVolts()` converte a contagem em volts usando o ganho; o sketch multiplica por 1,5 para desfazer o divisor.
- Repare na saída: `V_Po:2.5031`. Quatro casas decimais com sentido — no ADC do ESP32 a terceira casa já era ruído.

Experimento: ESP32 × ADS1115

1. Grave `01b` (ADC interno), sonda num tampão pH 7, anote 20 leituras de tensão e calcule a diferença entre a maior e a menor.
2. Grave `01c` (ADS1115) e repita. Quantas vezes menor ficou o espalhamento?
3. Com o ADC interno, aproxime o celular (Wi-Fi em uso, baixando algo) do ESP32: as leituras mudam? E com o ADS1115?
4. Meça o mesmo tampão pH 4 nos dois: qual dos dois precisou de menos “correção” na calibração (compare a inclinação com a teórica −0,177 V/pH)?

Parte 5 — Projeto: monitor de água

Agora juntamos os três sensores, um **LED RGB** e um **buzzer**. O monitor acompanha a água de um reservatório (um balde ou uma garrafa grande com uma torneira de barril, por exemplo): mede o pH, conta quanto sai pela torneira, mostra o nível e **age**: avisa quando o pH sai da faixa, quando alguém esquece a torneira aberta (vazamento), quando o consumo passa da meta do dia e quando o reservatório está transbordando ou quase vazio.

LED	Significado	Buzzer
Branco rápido ao ligar	teste	1 bipe
Verde	tudo normal	—
Azul	água passando pelo sensor de vazão	1 bipe a cada litro
Amarelo	pH a menos de 0,5 do limite, ou reservatório abaixo de 5 cm	—
Vermelho fixo	pH fora de 6,0–9,0 (com histerese de 0,2) ou transbordando (> 25 cm)	2 bipes ao sair da faixa; sirene no transbordo
Vermelho piscando	**vazamento**: água correndo há mais de 5 min sem parar	sirene
(qualquer)	consumo passou de 50 L	3 bipes, uma vez

Lógica que vale a pena entender

Histerese: o pH que oscila em 5,99 / 6,01 faria o alarme ligar e desligar sem parar. O sketch exige que o pH passe de 6,0 para alarmar e **volte acima de 6,2** para desarmar.

Vazamento é definido como **tempo**: água correndo (≥ 0,3 L/min) há mais de `TEMPO_VAZAMENTO_S`. Para testar na aula, mude para 20 s.

Prioridade: vazamento > transbordo > pH ruim > nível baixo/atenção > água passando > ok. O LED mostra sempre o mais grave.

No ESP32 os sons são feitos **sem `delay()`** (`atenderSom()` toca um pedacinho a cada volta do `loop()`): a página do celular não pode travar enquanto a sirene toca.

5A — Monitor no Arduino Uno

Peça	Ligação no Uno
Trilhas da protoboard	+ = 5V do Uno, − = GND
pH-4502C	V+ no +, G no −, Po no **A0**
YF-S201	vermelho no +, preto no −, amarelo no **D2**
HX710B	VCC no +, GND no −, OUT no **D3**, SCK no **D4**
LED RGB (cátodo comum)	R → 220 Ω → **D9**, G → 220 Ω → **D6**, B → 220 Ω → **D5**, cátodo no −
Buzzer passivo	+ no **D8**, − no −

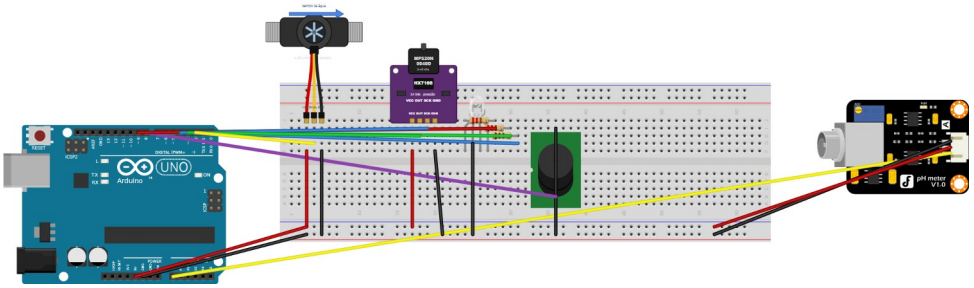


Figura 8 — Parte 5A — monitor completo no Uno: YF-S201, HX710B, LED RGB, buzzer e a placa de pH.

1. Monte **um sensor por vez** e teste com o sketch individual antes de passar ao próximo. As calibrações feitas com os sketches 01, 02 e 03 **ficam na EEPROM** nos mesmos endereços: o monitor já nasce calibrado.
2. Grave `04_monitor_agua_uno`, abra o Monitor Serial: uma linha por segundo com pH, vazão, volume e nível.
3. Teste cada alarme: vinagre na água (pH), torneira aberta por 20 s com `TEMPO_VAZAMENTO_S = 20` (vazamento), encha o reservatório além de 25 cm (transbordo), digite `z` e deixe passar 2 L (bipes de consumo).

04_monitor_agua_uno.ino

```
/*
  04 - PROJETO: Monitor de agua no Arduino Uno - pH + vazao + nivel + LED RGB + buzzer
  Placa: Arduino Uno     Sem biblioteca (EEPROM para as calibracoes)

  Ligacoes (protoboard; trilha + = 5 V do Uno, trilha - = GND):
    pH-4502C        V+ -> 5V   G -> GND   Po -> A0
    YF-S201         vermelho -> 5V   preto -> GND   amarelo -> D2 (INT0)
    HX710B          VCC -> 5V   OUT -> D3   SCK -> D4   GND -> GND   (mangueira ate o tubo no
reservatorio)
    LED RGB (catodo comum)  R -> 220 R -> D9   G -> 220 R -> D6   B -> 220 R -> D5   catodo ->
GND
    Buzzer passivo  + -> D8   - -> GND

  O que o monitor faz (uma leitura por segundo):
    - mede pH, vazao (acumulando o volume) e o nivel do reservatorio
    - LED:   VERDE = tudo bem | AZUL = agua passando | AMARELO = pH perto do limite ou
reservatorio baixo |
             VERMELHO = pH fora de 6,0-9,0 ou transbordando | VERMELHO PISCANDO = VAZAMENTO (agua
correndo ha mais de TEMPO_VAZAMENTO)
    - buzzer: 2 bipes quando o pH sai da faixa (uma vez, com HISTERESE de 0,2) | sirene no
vazamento e no transbordo |
              1 bipe a cada litro que passa | 3 bipes ao passar o LIMITE_DIA
  Comandos no Serial (115200): c7 c4 c10 (pH)  z (zera volume)  b / f <litros> (K)  c0 / c <cm>
(nivel)  s (silencia)
  As calibracoes ficam na EEPROM nos MESMOS enderecos dos sketches 01, 02 e 03: se voce calibrou
com eles
  nesta placa, o monitor ja comeca calibrado.
*/
#include <EEPROM.h>

// ------------------------------------------------------------------ pinos e ajustes
const int PINO_PH = A0, PINO_VAZAO = 2, PINO_OUT = 3, PINO_SCK = 4, LED_R = 9, LED_G = 6, LED_B =
5, BUZZER = 8;
const float VREF = 5.0;
const float PH_MIN = 6.0, PH_MAX = 9.0, PH_ATENCAO = 0.5, HISTERESE = 0.2;
const unsigned long TEMPO_VAZAMENTO_S = 300;     // agua correndo sem parar por 5 min = vazamento
(para testar: 20)
const float LIMITE_DIA_L = 50.0;                 // consumo que dispara o aviso
const float VAZAO_MINIMA = 0.3;                  // L/min abaixo disso = "parado"
const float NIVEL_MIN_CM = 5.0, NIVEL_MAX_CM = 25.0;   // reservatorio da aula (ajuste)
bool silenciado = false;

// ------------------------------------------------------------------ pH (sketch 01)
float V7 = 2.50, INCLINACAO = -0.177;
float lerTensao() {
  const int N = 30; int v[N];
```

```
  for (int i = 0; i < N; i++) { v[i] = analogRead(PINO_PH); delay(2); }
  for (int i = 0; i < N - 1; i++) for (int j = 0; j < N - 1 - i; j++) if (v[j] > v[j + 1]) { int
t = v[j]; v[j] = v[j + 1]; v[j + 1] = t; }
  long soma = 0; for (int i = 5; i < N - 5; i++) soma += v[i];
  return (soma / float(N - 10)) * VREF / 1023.0;
}
float tensaoParaPH(float v) { return 7.0 + (v - V7) / INCLINACAO; }

// ---------------------------------------------------------------- vazao (sketch 02)
float K = 7.5;
volatile unsigned long pulsos = 0;
void contarPulso() { pulsos++; }
unsigned long pulsosCalibracao = 0; bool calibrando = false;
float volumeL = 0, vazao = 0; int litrosAvisados = 0;
unsigned long correndoDesde = 0;

// ---------------------------------------------------------------- nivel (sketch 03)
long zero = 0; float sens = 50000.0;
long lerHX710B() {
  unsigned long t0 = millis();
  while (digitalRead(PINO_OUT) == HIGH) { if (millis() - t0 > 300) return 0x7FFFFFFF; }
  long v = 0;
  noInterrupts();
  for (int i = 0; i < 24; i++) { digitalWrite(PINO_SCK, HIGH); delayMicroseconds(1); v = (v << 1)
| digitalRead(PINO_OUT); digitalWrite(PINO_SCK, LOW); delayMicroseconds(1); }
  digitalWrite(PINO_SCK, HIGH); delayMicroseconds(1); digitalWrite(PINO_SCK, LOW);
  interrupts();
  if (v & 0x800000L) v |= 0xFF000000L;
  return v;
}
float lerNivelCm() {                          // 3 leituras (~0,3 s); NAN se o modulo nao
responder
  float soma = 0; int ok = 0;
  for (int i = 0; i < 3; i++) { long v = lerHX710B(); if (v != 0x7FFFFFFF) { soma += v; ok++; } }
  return ok ? (soma / ok - zero) / sens : NAN;
}

// ---------------------------------------------------------------- EEPROM: 0 V7 | 4 incl | 8 K
| 12 zero | 16 sens
void salvar()   { EEPROM.put(0, V7); EEPROM.put(4, INCLINACAO); EEPROM.put(8, K); EEPROM.put(12,
zero); EEPROM.put(16, sens); }
void carregar() {
  float a, b, k, s; long z; EEPROM.get(0, a); EEPROM.get(4, b); EEPROM.get(8, k); EEPROM.get(12,
z); EEPROM.get(16, s);
  if (!isnan(a) && a > 1 && a < 4) V7 = a;
  if (!isnan(b) && b < -0.05 && b > -0.5) INCLINACAO = b;
  if (!isnan(k) && k > 3 && k < 15) K = k;
  if (z != -1 && z > -8000000L && z < 8000000L) zero = z;   // -1 = EEPROM virgem (0xFFFFFFFF)
  if (!isnan(s) && s > 100 && s < 1e7) sens = s;
}

// ---------------------------------------------------------------- LED e buzzer
void cor(bool r, bool g, bool b) { digitalWrite(LED_R, r); digitalWrite(LED_G, g);
digitalWrite(LED_B, b); }
void bipes(int n, int freqHz, int ms) {
  if (silenciado) return;
  for (int i = 0; i < n; i++) { tone(BUZZER, freqHz, ms); delay(ms + 60); }
}
void sirene() {                               // ~0,7 s de sirene a cada leitura enquanto
durar o alarme
  if (silenciado) return;
  for (int f = 600; f < 1500; f += 50) { tone(BUZZER, f, 20); delay(20); }
  for (int f = 1500; f > 600; f -= 50) { tone(BUZZER, f, 20); delay(20); }
}

// ---------------------------------------------------------------- programa
bool phForaDaFaixa = false, vazamento = false, limiteAvisado = false, pisca = false;
unsigned long ultimaLeitura = 0;


void setup() {
  Serial.begin(115200);
  Serial.setTimeout(50);              // comandos chegam inteiros; sem isto readStringUntil
espera 1 s
  pinMode(LED_R, OUTPUT); pinMode(LED_G, OUTPUT); pinMode(LED_B, OUTPUT); pinMode(BUZZER,
OUTPUT);
  pinMode(PINO_VAZAO, INPUT_PULLUP);
```

```
  pinMode(PINO_OUT, INPUT); pinMode(PINO_SCK, OUTPUT); digitalWrite(PINO_SCK, LOW);
  attachInterrupt(digitalPinToInterrupt(PINO_VAZAO), contarPulso, RISING);
  carregar();
  cor(1, 1, 1); bipes(1, 2000, 80); delay(200); cor(0, 0, 0);
  Serial.println(F("Monitor de agua: pH + vazao + nivel. Comandos: c7 c4 c10 z b f<L> c0 c<cm>
s"));
  Serial.print(F("V7=")); Serial.print(V7, 3); Serial.print(F(" incl="));
Serial.print(INCLINACAO, 4);
  Serial.print(F(" K=")); Serial.print(K, 3); Serial.print(F(" zero=")); Serial.print(zero);
Serial.print(F(" sens=")); Serial.println(sens, 0);
}

void loop() {
  if (Serial.available()) {
    String cmd = Serial.readStringUntil('\n'); cmd.trim();
    if (cmd == "c7") { V7 = lerTensao(); salvar(); Serial.println(F("V7 gravado")); }
    else if (cmd == "c4" || cmd == "c10") { INCLINACAO = (lerTensao() - V7) / ((cmd == "c4" ? 4.0
: 10.0) - 7.0); salvar(); Serial.print(F("inclinacao ")); Serial.println(INCLINACAO, 4); }
    else if (cmd == "z") { volumeL = 0; litrosAvisados = 0; limiteAvisado = false;
Serial.println(F("volume zerado")); }
    else if (cmd == "b") { calibrando = true; pulsosCalibracao = 0; Serial.println(F("calibrando:
encha o recipiente e digite f <litros>")); }
    else if (cmd.startsWith("f") && calibrando) {
      float litros = cmd.substring(1).toFloat();
      if (litros > 0 && pulsosCalibracao > 50) { K = (pulsosCalibracao / litros) / 60.0;
salvar(); calibrando = false; Serial.print(F("K = ")); Serial.println(K, 3); }
    }
    else if (cmd == "c0") { long v = lerHX710B(); if (v != 0x7FFFFFFF) { zero = v; salvar();
Serial.print(F("zero gravado: ")); Serial.println(zero); } }
    else if (cmd.startsWith("c ")) {
      float cm = cmd.substring(2).toFloat(); long v = lerHX710B();
      if (cm > 0 && v != 0x7FFFFFFF && labs(v - zero) > 100) { sens = (v - zero) / cm; salvar();
Serial.print(F("sens = ")); Serial.println(sens, 1); }
    }
    else if (cmd == "s") { silenciado = !silenciado; Serial.println(silenciado ? F("buzzer
silenciado") : F("buzzer ligado")); }
  }

  if (millis() - ultimaLeitura < 1000) return;
  unsigned long agora = millis();
  float dt = (agora - ultimaLeitura) / 1000.0;
  ultimaLeitura = agora;

  // --- vazao e volume
  noInterrupts(); unsigned long n = pulsos; pulsos = 0; interrupts();
  vazao = (n / dt) / K;
  volumeL += n / (K * 60.0);
  if (calibrando) pulsosCalibracao += n;
  bool correndo = vazao >= VAZAO_MINIMA;
  if (correndo && correndoDesde == 0) correndoDesde = agora;
  if (!correndo) correndoDesde = 0;
  vazamento = correndo && (agora - correndoDesde) / 1000 >= TEMPO_VAZAMENTO_S;

  // --- pH com histerese
  float ph = tensaoParaPH(lerTensao());
  bool foraAgora = phForaDaFaixa ? (ph < PH_MIN + HISTERESE || ph > PH_MAX - HISTERESE) : (ph <
PH_MIN || ph > PH_MAX);
  if (foraAgora && !phForaDaFaixa) bipes(2, 2000, 80);
  phForaDaFaixa = foraAgora;
  bool atencao = !foraAgora && (ph < PH_MIN + PH_ATENCAO || ph > PH_MAX - PH_ATENCAO);

  // --- nivel
  float nivel = lerNivelCm();
  bool transbordo = !isnan(nivel) && nivel > NIVEL_MAX_CM;
  bool baixo = !isnan(nivel) && nivel < NIVEL_MIN_CM;

  // --- consumo
  if ((int)volumeL > litrosAvisados) { litrosAvisados = (int)volumeL; bipes(1, 1500, 40); }
  if (volumeL >= LIMITE_DIA_L && !limiteAvisado) { limiteAvisado = true; bipes(3, 2500, 150); }

  // --- LED (prioridade: vazamento > transbordo > pH ruim > nivel baixo / atencao > agua
passando > ok)
  pisca = !pisca;
  if (vazamento)              { cor(pisca, 0, 0); sirene(); }
  else if (transbordo)        { cor(1, 0, 0); sirene(); }
  else if (phForaDaFaixa)     cor(1, 0, 0);
```

```
  else if (baixo || atencao)   cor(1, 1, 0);
  else if (correndo)           cor(0, 0, 1);
  else                         cor(0, 1, 0);

  // --- tabela
  Serial.print(F("pH:")); Serial.print(ph, 2);
  Serial.print(F(" Vazao_L_min:")); Serial.print(vazao, 2);
  Serial.print(F(" Volume_L:")); Serial.print(volumeL, 3);
  Serial.print(F(" Nivel_cm:")); if (isnan(nivel)) Serial.print(F("--")); else
Serial.print(nivel, 1);
  if (correndo) { Serial.print(F(" correndo_s:")); Serial.print((agora - correndoDesde) /
1000); }
  if (vazamento) Serial.print(F("  <<< VAZAMENTO"));
  if (transbordo) Serial.print(F("  <<< TRANSBORDANDO"));
  if (baixo) Serial.print(F("  <<< reservatorio baixo"));
  if (phForaDaFaixa) Serial.print(F("  <<< pH fora de 6,0-9,0"));
  if (calibrando) { Serial.print(F("  [calibrando: ")); Serial.print(pulsosCalibracao);
Serial.print(F("]")); }
  Serial.println();
}
```

5B — Monitor no ESP32 com página no celular

Peça	Ligação no ESP32
Trilha + (baixo)	**VIN (5 V)**; trilha − = GND
pH-4502C	V+ no +, G no −, Po → 1 kΩ → **GPIO 34** → 2 kΩ → −
YF-S201	vermelho no +, preto no −, amarelo → 1 kΩ → **GPIO 27** → 2 kΩ → −
HX710B	VCC no **3V3**, GND no −, OUT no **GPIO 18**, SCK no **GPIO 19**
LED RGB	R → 220 Ω → **GPIO 25**, G → 220 Ω → **GPIO 26**, B → 220 Ω → **GPIO 33**, cátodo no −
Buzzer passivo	+ no **GPIO 14**, − no −

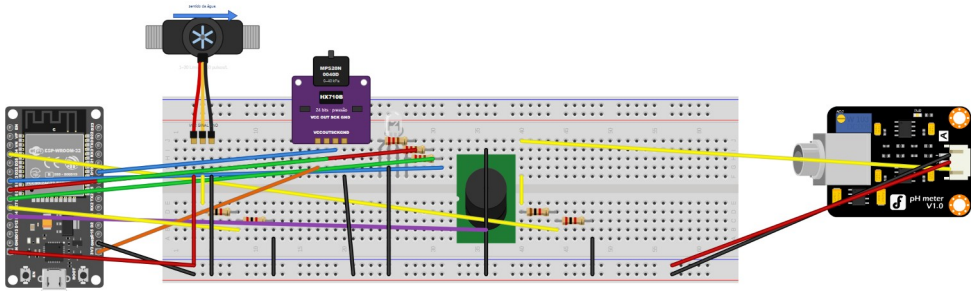


Figura 9 — Parte 5B — monitor completo no ESP32: divisores do YF-S201 e do pH na metade de baixo da protoboard, HX710B no 3V3, LED RGB e buzzer.

O sketch do ESP32 cria a rede Wi-Fi **Agua-Monitor** (senha `12345678`). Conecte o celular a ela (desligue os dados móveis) e abra **http://192.168.4.1**: a página mostra o valor principal, os outros valores em cartões, um gráfico das últimas leituras e um aviso de **ALERTA**. Os arquivos `monitor_wifi.h` e `pagina_web.h` precisam estar **na mesma pasta** do sketch (a IDE os abre como abas).

As calibrações do ESP32 ficam na **Preferences** nos mesmos espaços dos sketches `01b` (`ph`), `02b` (`vazao`) e `03b` (`nivel`): calibre com os sketches individuais **nesta placa e nesta montagem** (divisores, 3,3 V no HX710B) e o monitor já nasce calibrado. Se o HX710B não responder, a página mostra **−1 cm** e um alerta, em vez de um falso “0 cm”.

04b_monitor_agua_esp32_web.ino

```
/*
  04b - PROJETO: Monitor de agua no ESP32 - pH + vazao + nivel + LED RGB + buzzer + pagina no
celular
  Placa: ESP32 Dev Module     Sem biblioteca (Preferences)     Arquivos extras: monitor_wifi.h,
pagina_web.h

  Ligacoes (protoboard 830; trilha + = VIN (5 V), trilha - = GND):
    pH-4502C        V+ -> VIN   G -> GND   Po -> 1 k -> GPIO 34 -> 2 k -> GND   (divisor)
    YF-S201         vermelho -> VIN   preto -> GND   amarelo -> 1 k -> GPIO 27 -> 2 k -> GND
(divisor)
    HX710B          VCC -> 3V3   OUT -> GPIO 18   SCK -> GPIO 19   GND -> GND   (sem divisor:
alimentado em 3,3 V)
                    ATENCAO: em 3,3 V a ponte do sensor recebe 3,3 V (nao 5 V) e a sensibilidade
cai ~1/3:
                    calibre o nivel (c0, c <cm>) NESTA montagem; a calibracao feita no Uno nao
vale aqui.
    LED RGB         R -> 220 R -> GPIO 25   G -> 220 R -> GPIO 26   B -> 220 R -> GPIO 33
catodo -> GND
    Buzzer passivo  + -> GPIO 14   - -> GND
  Mesma logica do sketch 04 (Uno): LED de status, bipes do pH com histerese, sirene de
vazamento/transbordo,
  aviso de consumo. Diferencas: o loop() nunca bloqueia (sons por etapas, com millis()) e tudo
vai para o celular.
  Comandos no Serial (115200): c7 c4 c10 z b f<L> c0 c<cm> s.    Celular: rede "Agua-Monitor" ->
http://192.168.4.1
*/
#include <Preferences.h>
#include "monitor_wifi.h"

const int PINO_PH = 34, PINO_VAZAO = 27, PINO_OUT = 18, PINO_SCK = 19, LED_R = 25, LED_G = 26,
LED_B = 33, BUZZER = 14;
const float PH_MIN = 6.0, PH_MAX = 9.0, PH_ATENCAO = 0.5, HISTERESE = 0.2;
const unsigned long TEMPO_VAZAMENTO_S = 300;
const float LIMITE_DIA_L = 50.0, VAZAO_MINIMA = 0.3;
const float NIVEL_MIN_CM = 5.0, NIVEL_MAX_CM = 25.0;
const float V7_PADRAO = 1.667, INCL_PADRAO = -0.118, K_PADRAO = 7.5, SENS_PADRAO = 50000.0;
float V7 = V7_PADRAO, INCLINACAO = INCL_PADRAO, K = K_PADRAO, sens = SENS_PADRAO; long zero = 0;
bool silenciado = false;
Preferences prefs;

// ------------------------------------------------------------------ sensores
float lerTensao() {
  const int N = 30; int v[N];
  for (int i = 0; i < N; i++) { v[i] = analogReadMilliVolts(PINO_PH); delay(1); }
  for (int i = 0; i < N - 1; i++) for (int j = 0; j < N - 1 - i; j++) if (v[j] > v[j + 1]) { int
t = v[j]; v[j] = v[j + 1]; v[j + 1] = t; }
  long soma = 0; for (int i = 5; i < N - 5; i++) soma += v[i];
  return (soma / float(N - 10)) / 1000.0;
}
float tensaoParaPH(float v) { return 7.0 + (v - V7) / INCLINACAO; }

volatile unsigned long pulsos = 0;
portMUX_TYPE mux = portMUX_INITIALIZER_UNLOCKED;
void IRAM_ATTR contarPulso() { portENTER_CRITICAL_ISR(&mux); pulsos++;
portEXIT_CRITICAL_ISR(&mux); }

long lerHX710B() {
  unsigned long t0 = millis();
  while (digitalRead(PINO_OUT) == HIGH) { if (millis() - t0 > 300) return 0x7FFFFFFF; yield(); }
  long v = 0;
  portDISABLE_INTERRUPTS();
  for (int i = 0; i < 24; i++) { digitalWrite(PINO_SCK, HIGH); delayMicroseconds(1); v = (v << 1)
| digitalRead(PINO_OUT); digitalWrite(PINO_SCK, LOW); delayMicroseconds(1); }
  digitalWrite(PINO_SCK, HIGH); delayMicroseconds(1); digitalWrite(PINO_SCK, LOW);
  portENABLE_INTERRUPTS();
  if (v & 0x800000L) v |= 0xFF000000L;
  return v;
}
float lerNivelCm() { long v = lerHX710B(); return v == 0x7FFFFFFF ? NAN : (v - zero) /
sens; }   // 1 leitura/s: media feita pelo sensor (10 Hz)

// Calibracoes nos MESMOS namespaces dos sketches 01b ("ph"), 02b ("vazao") e 03b ("nivel"): se
voce calibrou com eles
// neste ESP32, o monitor ja comeca calibrado.
void salvar() {
```

```
  prefs.begin("ph", false);    prefs.putFloat("v7", V7); prefs.putFloat("incl", INCLINACAO);
prefs.end();
  prefs.begin("vazao", false); prefs.putFloat("k", K); prefs.end();
  prefs.begin("nivel", false); prefs.putLong("zero", zero); prefs.putFloat("sens", sens);
prefs.end();
}
void carregar() {
  prefs.begin("ph", true);    V7 = prefs.getFloat("v7", V7); INCLINACAO = prefs.getFloat("incl",
INCLINACAO); prefs.end();
  prefs.begin("vazao", true); K = prefs.getFloat("k", K); prefs.end();
  prefs.begin("nivel", true); zero = prefs.getLong("zero", 0); sens = prefs.getFloat("sens",
sens); prefs.end();
  if (K < 3 || K > 15) K = K_PADRAO;
  if (sens < 100 || sens > 1e7) sens = SENS_PADRAO;
}

// ------------------------------------------------------------------ LED e buzzer sem bloquear
void cor(bool r, bool g, bool b) { digitalWrite(LED_R, r); digitalWrite(LED_G, g);
digitalWrite(LED_B, b); }
int bipesPendentes = 0; unsigned long proximoBipe = 0; int bipeFreq = 2000, bipeMs = 80;
void bipes(int n, int freqHz, int ms) { if (silenciado) return; bipesPendentes = n; bipeFreq =
freqHz; bipeMs = ms; proximoBipe = 0; }
bool sireneLigada = false; unsigned long ultimoPassoSirene = 0; int freqSirene = 600, passoSirene
= 50;
void atenderSom() {
  unsigned long agora = millis();
  if (sireneLigada && !silenciado) {                          // varredura 600 -> 1500 -> 600
Hz, 20 ms por passo
    if (agora - ultimoPassoSirene >= 20) { ultimoPassoSirene = agora; tone(BUZZER, freqSirene,
25); freqSirene += passoSirene; if (freqSirene >= 1500 || freqSirene <= 600) passoSirene =
-passoSirene; }
    return;
  }
  if (bipesPendentes > 0 && agora >= proximoBipe) { tone(BUZZER, bipeFreq, bipeMs); proximoBipe =
agora + bipeMs + 60; bipesPendentes--; }
}

// ------------------------------------------------------------------ programa
float ph = 7, vazao = 0, volumeL = 0, nivel = NAN; int litrosAvisados = 0;
unsigned long correndoDesde = 0, ultimaLeitura = 0, pulsosCalibracao = 0;
bool calibrando = false, phForaDaFaixa = false, vazamento = false, transbordo = false, baixo =
false, limiteAvisado = false, pisca = false;

void setup() {
  Serial.begin(115200);
  Serial.setTimeout(50);              // comandos chegam inteiros; sem isto readStringUntil
espera 1 s
  analogReadResolution(12); analogSetAttenuation(ADC_11db);
  pinMode(LED_R, OUTPUT); pinMode(LED_G, OUTPUT); pinMode(LED_B, OUTPUT); pinMode(BUZZER,
OUTPUT);
  pinMode(PINO_VAZAO, INPUT);
  pinMode(PINO_OUT, INPUT); pinMode(PINO_SCK, OUTPUT); digitalWrite(PINO_SCK, LOW);
  attachInterrupt(digitalPinToInterrupt(PINO_VAZAO), contarPulso, RISING);
  carregar();
  cor(1, 1, 1); tone(BUZZER, 2000, 100); delay(200); cor(0, 0, 0);
  Serial.printf("Monitor de agua. V7=%.3f incl=%.4f K=%.3f zero=%ld sens=%.0f. Comandos: c7 c4
c10 z b f<L> c0 c<cm> s\n", V7, INCLINACAO, K, zero, sens);
  monitorIniciar("Agua-Monitor");
}

void loop() {
  monitorAtender();
  atenderSom();
  if (Serial.available()) {
    String cmd = Serial.readStringUntil('\n'); cmd.trim();
    if (cmd == "c7") { V7 = lerTensao(); salvar(); Serial.println("V7 gravado"); }
    else if (cmd == "c4" || cmd == "c10") { INCLINACAO = (lerTensao() - V7) / ((cmd == "c4" ? 4.0
: 10.0) - 7.0); salvar(); Serial.printf("inclinacao %.4f\n", INCLINACAO); }
    else if (cmd == "z") { volumeL = 0; litrosAvisados = 0; limiteAvisado = false;
Serial.println("volume zerado"); }
    else if (cmd == "b") { calibrando = true; pulsosCalibracao = 0; Serial.println("calibrando:
encha o recipiente e digite f <litros>"); }
    else if (cmd.startsWith("f") && calibrando) {
      float litros = cmd.substring(1).toFloat();
      if (litros > 0 && pulsosCalibracao > 50) { K = (pulsosCalibracao / litros) / 60.0;
salvar(); calibrando = false; Serial.printf("K = %.3f\n", K); }
    }
```

```
    else if (cmd == "c0") { long v = lerHX710B(); if (v != 0x7FFFFFFF) { zero = v; salvar();
Serial.printf("zero gravado: %ld\n", zero); } }
    else if (cmd.startsWith("c ")) {
      float cm = cmd.substring(2).toFloat(); long v = lerHX710B();
      if (cm > 0 && v != 0x7FFFFFFF && labs(v - zero) > 100) { sens = (v - zero) / cm; salvar();
Serial.printf("sens = %.1f\n", sens); }
    }
    else if (cmd == "s") { silenciado = !silenciado; sireneLigada = false;
Serial.println(silenciado ? "buzzer silenciado" : "buzzer ligado"); }
  }

  if (millis() - ultimaLeitura < 1000) return;
  unsigned long agora = millis();
  float dt = (agora - ultimaLeitura) / 1000.0;
  ultimaLeitura = agora;

  portENTER_CRITICAL(&mux); unsigned long n = pulsos; pulsos = 0; portEXIT_CRITICAL(&mux);
  vazao = (n / dt) / K;
  volumeL += n / (K * 60.0);
  if (calibrando) pulsosCalibracao += n;
  bool correndo = vazao >= VAZAO_MINIMA;
  if (correndo && correndoDesde == 0) correndoDesde = agora;
  if (!correndo) correndoDesde = 0;
  vazamento = correndo && (agora - correndoDesde) / 1000 >= TEMPO_VAZAMENTO_S;

  ph = tensaoParaPH(lerTensao());
  bool foraAgora = phForaDaFaixa ? (ph < PH_MIN + HISTERESE || ph > PH_MAX - HISTERESE) : (ph <
PH_MIN || ph > PH_MAX);
  if (foraAgora && !phForaDaFaixa) bipes(2, 2000, 80);
  phForaDaFaixa = foraAgora;
  bool atencao = !foraAgora && (ph < PH_MIN + PH_ATENCAO || ph > PH_MAX - PH_ATENCAO);

  nivel = lerNivelCm();
  transbordo = !isnan(nivel) && nivel > NIVEL_MAX_CM;
  baixo = !isnan(nivel) && nivel < NIVEL_MIN_CM;
  sireneLigada = vazamento || transbordo;

  if ((int)volumeL > litrosAvisados) { litrosAvisados = (int)volumeL; bipes(1, 1500, 40); }
  if (volumeL >= LIMITE_DIA_L && !limiteAvisado) { limiteAvisado = true; bipes(3, 2500, 150); }

  pisca = !pisca;
  if (vazamento)              cor(pisca, 0, 0);
  else if (transbordo)        cor(1, 0, 0);
  else if (phForaDaFaixa)     cor(1, 0, 0);
  else if (baixo || atencao)  cor(1, 1, 0);
  else if (correndo)          cor(0, 0, 1);
  else                        cor(0, 1, 0);

  bool semNivel = isnan(nivel);                              // HX710B nao respondeu: a pagina
mostra -1 cm e avisa
  String msg = vazamento ? "VAZAMENTO: agua correndo ha " + String((agora - correndoDesde) /
60000) + " min"
             : transbordo ? "TRANSBORDANDO" : phForaDaFaixa ? "pH fora de 6,0-9,0" : baixo ?
"reservatorio baixo"
             : semNivel ? "sensor de nivel (HX710B) nao responde" : limiteAvisado ? "consumo
passou do limite" : "";
  Serial.printf("pH:%.2f Vazao_L_min:%.2f Volume_L:%.3f Nivel_cm:%.1f %s\n", ph, vazao, volumeL,
nivel, msg.c_str());
  monitorPublicar(jsonMonitor("Monitor de agua - pH, vazao e nivel",
    item("pH", ph, "") + "," + item("Vazao", vazao, "L/min") + "," + item("Volume", volumeL, "L")
+ "," + item("Nivel", semNivel ? -1 : nivel, "cm") + "," + item("Correndo ha", correndo ? (agora
- correndoDesde) / 1000 : 0, "s"),
    vazamento || transbordo || phForaDaFaixa || baixo || semNivel || limiteAvisado, msg));
}
```

E o caminho até a telemetria

O projeto da aula mostra os dados no celular. Para que eles apareçam no **painel da Rede de Telemetria** junto com os outros projetos da escola, o nó precisa publicar por **MQTT** com a biblioteca **CriaTelemetria** —

exatamente como fizemos na aula de qualidade do ar (sketch `06b`). O catálogo de tópicos já prevê, no projeto `horta`, as grandezas `ph` (pH), `vazao` (L/min), `pressao` (kPa) e `nivel`; o nosso HX710B entregaria a pressão em kPa, e o nível (em cm, em vez do estado da boia) seria uma grandeza nova a registrar no catálogo por *pull request*. É o **Desafio 6**.

Problemas comuns

Sintoma	Causa provável	Solução
pH sempre 7,00 (2,5 V fixo)	sonda desconectada ou BNC em curto	conferir o BNC; testar com os tampões
pH oscila ±0,5	sonda seca; BNC oxidado; água destilada (pouco condutiva); sem GND comum	hidratar 24 h em KCl; limpar o BNC; medir água da torneira
Inclinação < 0,15 V/pH (em módulo)	sonda velha ou suja	limpar (HCl 0,1 M por 10 min), reidratar; trocar
YF-S201: nenhum pulso	amarelo no pino errado (precisa ser 2 ou 3 no Uno); água no sentido contrário; vazão < 1 L/min	conferir o fio, a seta e abrir mais a torneira
YF-S201: vazão com saltos	pulsos perdidos (`delay()` longo no loop) ou ruído	nada de `delay()`; `INPUT_PULLUP`; cabo curto
YF-S201 no ESP32: zero sempre	sensor no 3V3 (mínimo 4,5 V) ou divisor invertido	VCC no VIN; conferir o divisor
HX710B: “não responde”	OUT/SCK trocados; sem VCC; SCK ficou em 1 (power-down)	conferir os fios; religar
HX710B: contagem “anda” com o tubo fora da água	deriva térmica; mangueira com água dentro	refazer `c0`; esvaziar a mangueira
HX710B: nível não muda	mangueira furada ou ponta do tubo fechada; ar escapando	conferir a vedação; a ponta do tubo deve ficar aberta
ADS1115: “não responde em 0x48”	SDA/SCL trocados; ADDR solto; VDD ausente	ADDR no GND; conferir SDA=21 SCL=22
ADS1115: leitura travada no máximo (32767)	entrada acima do fundo de escala (sem divisor)	divisor no Po ou ganho 2/3 (e VDD 5 V com conversor de nível)
ESP32 reinicia (brownout)	fonte fraca com Wi-Fi + sensores	fonte 5 V/2 A no VIN

Desafios

1. **Titulação assistida:** com o pH-4502C, adicione bicarbonato a vinagre gota a gota; o buzzer bipa a cada 0,5 de pH e o LED muda de cor ao cruzar pH 7. Registre `tempo,pH` e plote com o script Python da aula de distância: o ponto de viragem é onde a curva sobe mais depressa.
2. **Conta de água:** com o YF-S201 num chuveiro ou numa torneira, meça o volume de um banho ou de uma lavagem de louça. Com a tarifa da sua cidade, calcule o custo. Qual torneira gasta mais?
3. **Detector de vazamento melhor:** o YF-S201 não vê pingos. Combine com o HX710B: se o nível do reservatório cai sem que o sensor de vazão registre nada, há um vazamento por baixo (ou evaporação). Implemente e teste com uma garrafa furada.
4. **Volume pelo nível:** meça a área da base do seu reservatório e calcule o volume (V = A × h) só pelo HX710B. Compare com o volume somado pelo YF-S201 enquanto enche: a diferença é o erro combinado dos dois sensores.

5. **Compensação de temperatura do pH:** coloque o DS18B20 (aula de sensores) na amostra e corrija a inclinação: incl(T) = incl_{25} × (T + 273,15) / 298,15.
6. **Publicar na telemetria:** com a CriaTelemetria (aula de ar, sketch 06b), publique `ph`, `vazao` e `pressao` num nó do projeto `teste`. Depois, proponha no catálogo a grandeza `nivel` em `cm`.
7. **Leitura diferencial no ADS1115:** ligue o Po no A0 e o GND da placa de pH no A1 e leia o par **A0–A1** (`readADC_Differential_0_1`). O ruído de terra some? Compare.
8. **Pressostato:** com o HX710B e o módulo relé da aula Tapo, ligue uma bomba de aquário quando o nível cair abaixo de 10 cm e desligue acima de 20 cm — com histerese, como o nó da horta protege a bomba pela boia.

Referências

- Avia Semiconductor — *HX710A/HX710B 24-Bit ADC*, folha de dados (tabela de pulsos 25/26/27; 60 µs para power-down).
- Folha de dados MPS20N0040D (sensor de pressão piezorresistivo 0–40 kPa).
- Folhas de dados e listagens do YF-S201 / ZJ-S201 (F = 7,5 × Q; 450 pulsos/L; 5–18 V; ±10 %).
- Texas Instruments — *ADS111x Ultra-Small, Low-Power, I2C-Compatible, 16-Bit ADCs*, SBAS444.
- Eletrodo de pH E201 e módulo PH-4502C — notas do fabricante.
- Rede de Telemetria do Espaço CRIA — `telemetria-cria/firmware/horta-no1/README.md`, `mqtt/catalogo-topicos.md` (seção `horta/no1`), `calibracao/README.md`.